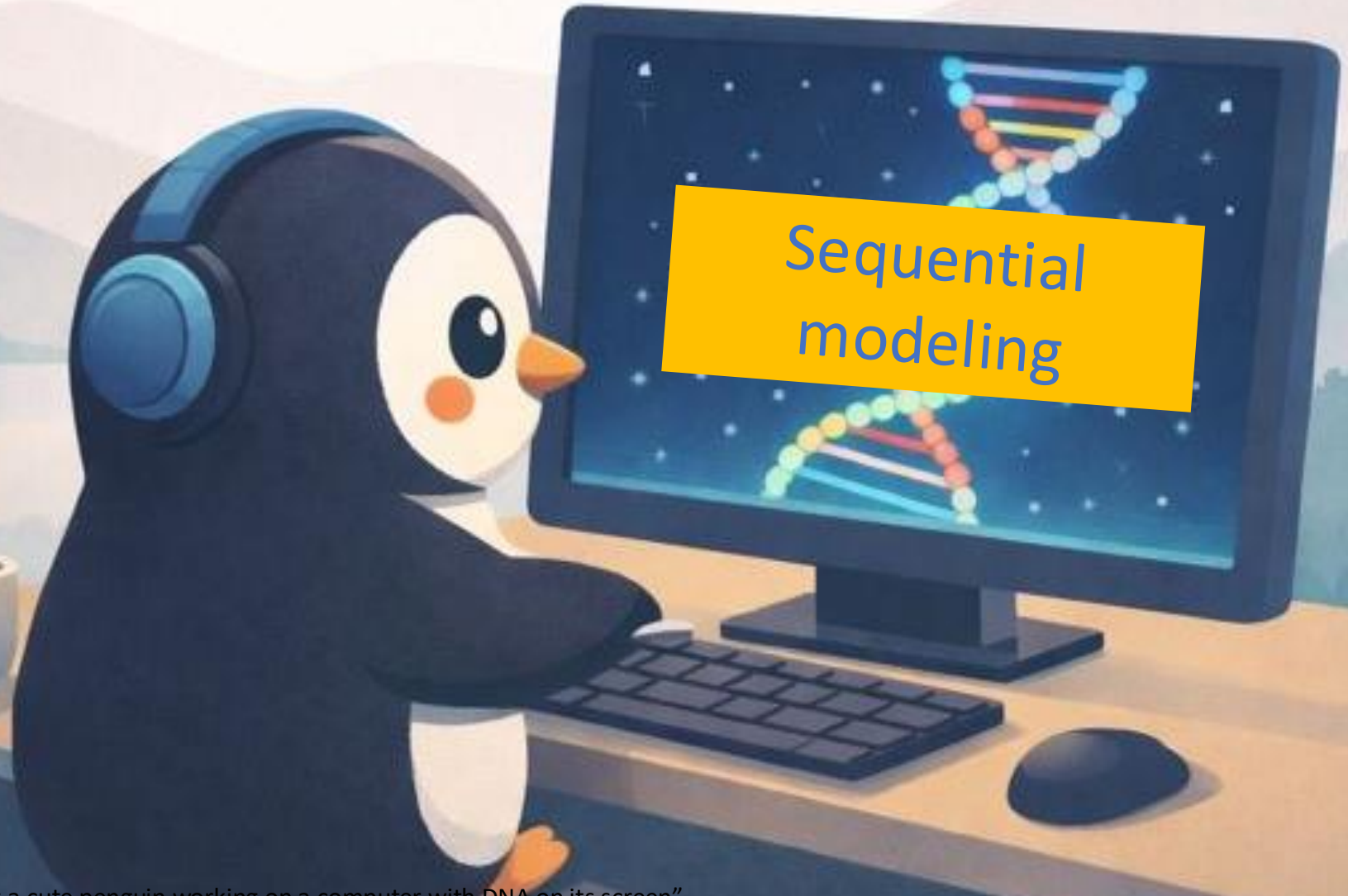


Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

April 14, 2026
Tuesday



GNN review

Inputs:
Node features +
Adjacency matrix

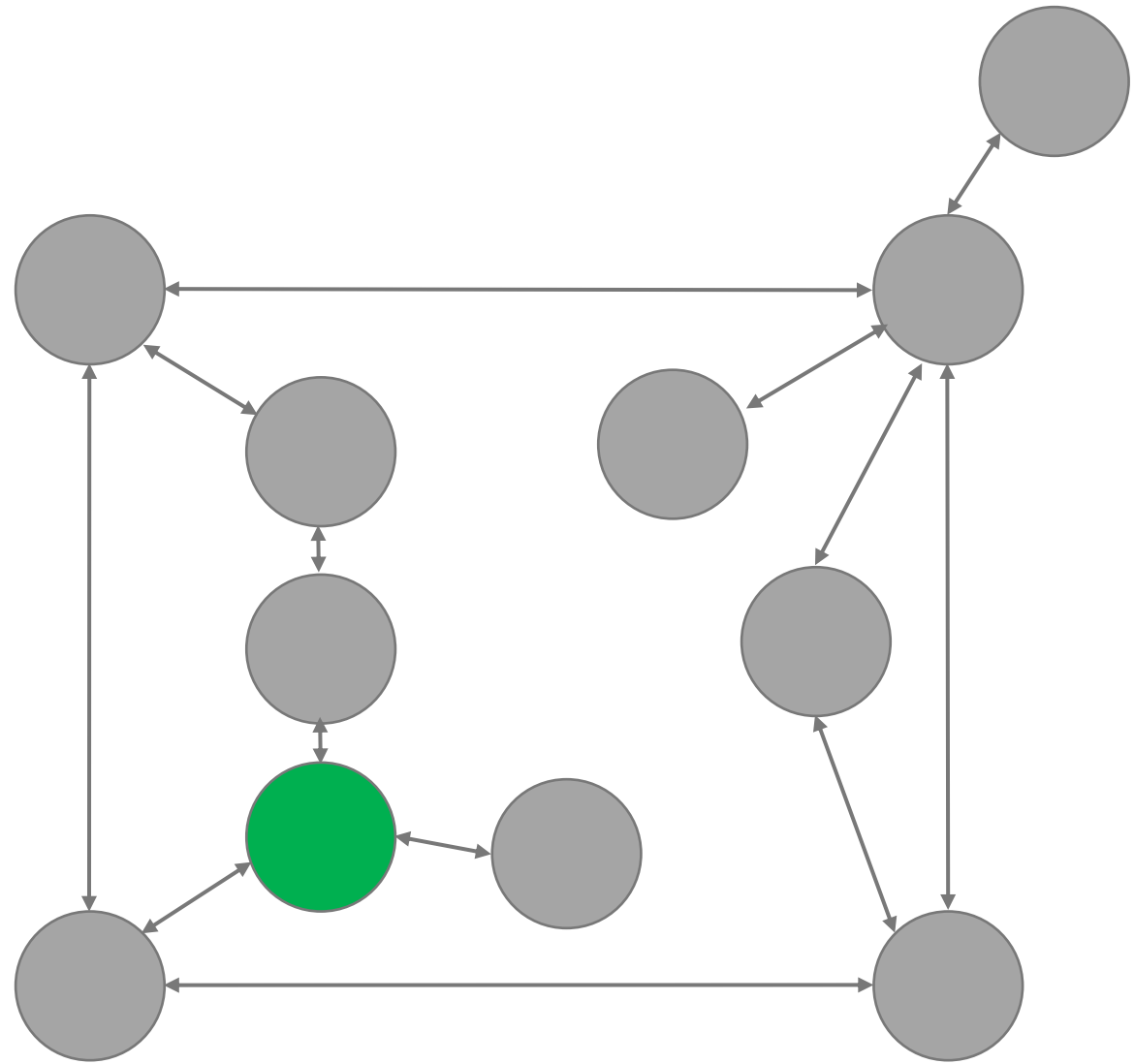


Image credit: [Medium](#)

Social Networks |

Node features: [24 (age), 1 (if employed),...]

Adjacency matrix:
$$\begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}$$



A more general graph

How do we define convolution on *this*?

Upcoming HW: Drug-based prediction tasks

Drug-disease prediction

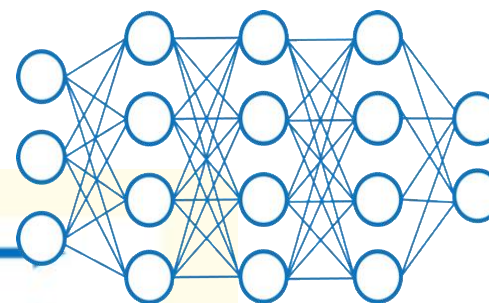


Drug-drug interaction prediction



Adverse Drug Reactions (ADRs) due to drug combinations

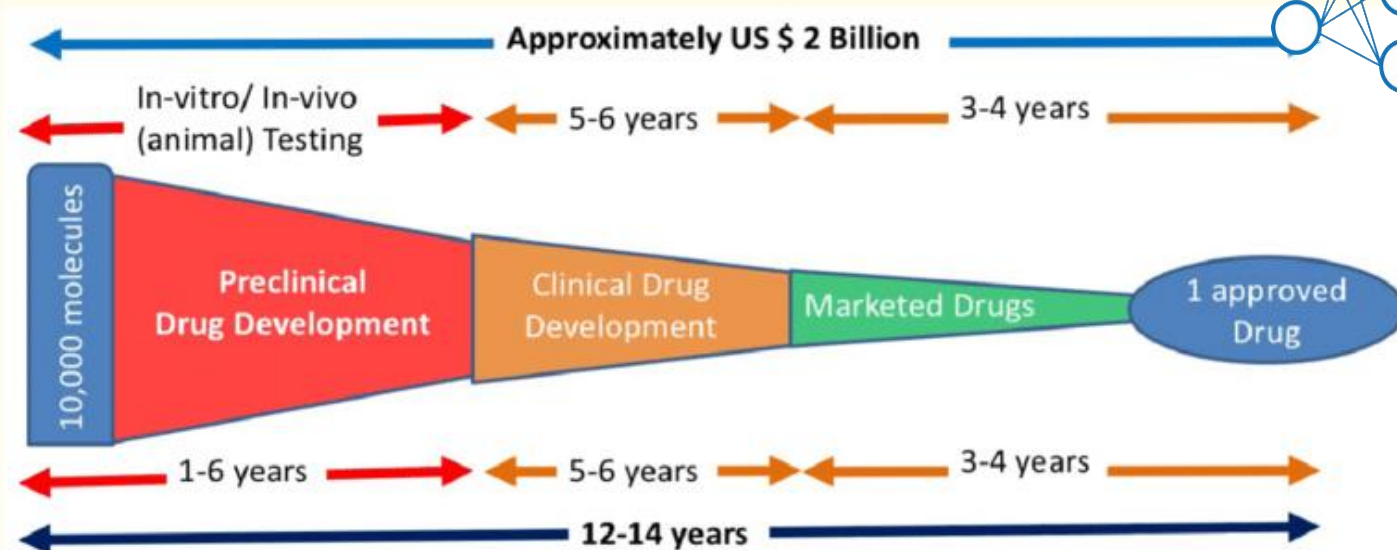
Deep Learning can assist by integrating existing databases!



DRUGBANK



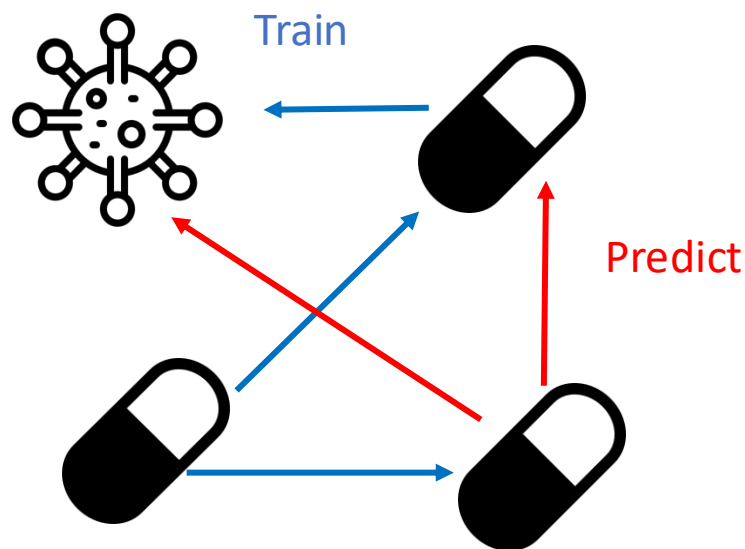
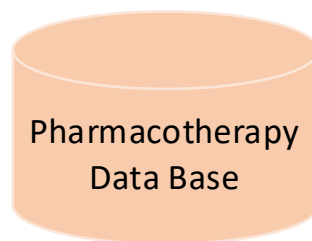
Pharmacotherapy Data Base



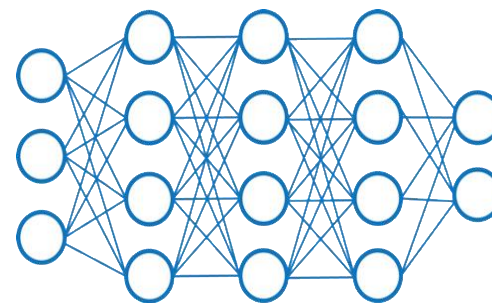
Drug development is time-intensive and costly

Graph Neural Networks (GNNs) for accurate supervised predictions

DRUGBANK



Supervised learning
EmerGNN, SumGNN and
KnowDDI



Graph Neural
Networks (GNNs)

Drug-disease prediction



Drug-drug interaction prediction



Today's goal – learn about sequential modeling and RNNs

(1) Language models

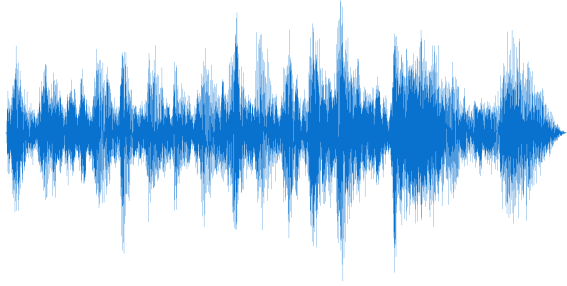
(2) Language embeddings

(3) Class activity: DNA-to-Vec

(4) N-gram language models

New data type: sequences

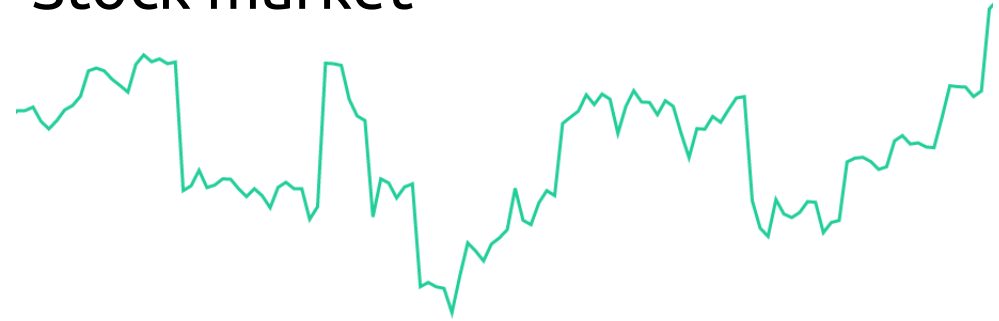
- Audio



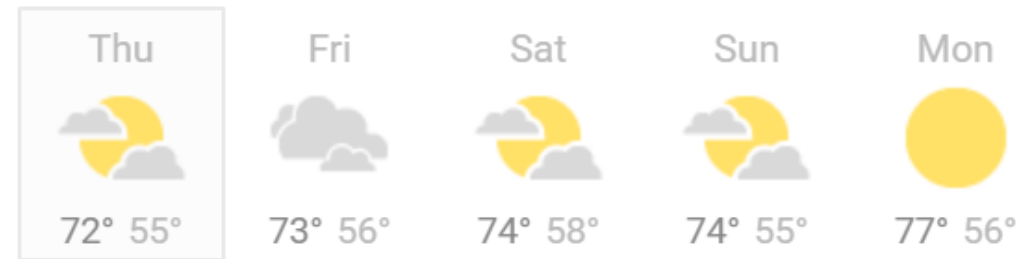
- DNA



- Stock market



- Weather



What is the data property here
that we could leverage?

Natural Language

“language that has developed naturally in use”

Natural Language

“language that has developed naturally in use”

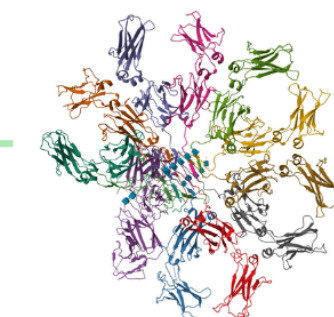
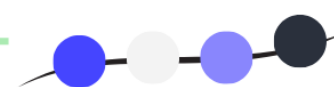
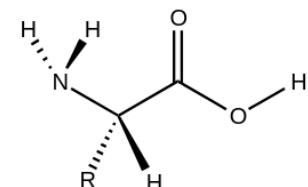
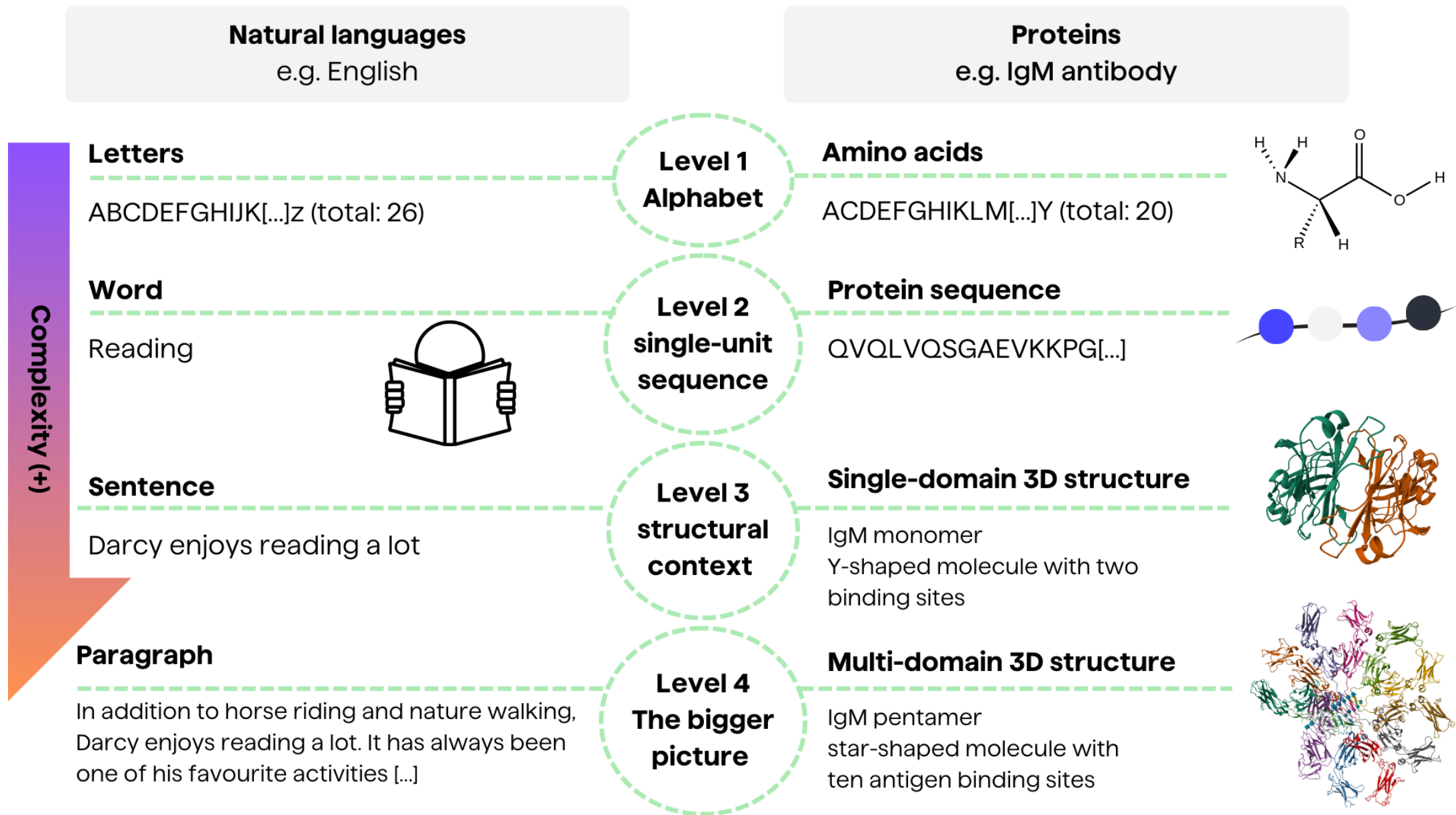
Compare to *constructed* or *formal* language

- code: `for i in range(50):`
- math: $52 + 94 = 147$
- logic: $A \wedge B \rightarrow C$ (if A and B, then C)

Natural Language

For ease of introduction to concepts: **sequence of *words***

“They went to the grocery store and bought bread, peanut butter, and jam.”



Natural Language: Prediction tasks?

Example of prediction?



Input: X

I do not want sour
cream in my
burrito



Function: f



Output: Y

No quiero crema
agrea en mi
burrito

Genomic/protein sequence??

Natural Language: Prediction tasks?

Example of classification?

Input: X

“The story telling was erratic and, at times, slow”

“Loved the diverse cast of this movie”



Function: f



Output: Y

“Good review?”



Genomic/protein sequence??

Natural Language: Prediction tasks?

Example of prediction?

*“They went to the grocery store and bought... bread?
milk?
rock?”*

Genomic/protein sequence??

Generating artificial sentences: Here each word is a discrete unit; predicting the next part of the sequence means predicting words

Language models

Language models

Definition: Probability distribution over strings in a language.

Exponentially-many strings means each string has very low probability

Relative probabilities are meaningful:

$P(\text{“they went to the store”}) \gg P(\text{“butter dancing rock”})$

Language models logic: leverage sentence structure

P(any sequence) is determined by **P(the words in the sequence)**.

Said differently, we can represent a sequence as $w_1, w_2, \dots w_n$, and

$$P(w_1, w_2, \dots w_n) = P(w_1) * P(w_2|w_1) * P(w_3|w_1, w_2) * \dots P(w_n|w_1 \dots w_{n-1})$$

$$P(\text{"they went to the store"}) = P(\text{"they"}) * P(\text{"went"} | \text{"they"}) * P(\text{"to"} | \text{"they went"}) * \dots$$

"The probability of a sentence is the product of the probabilities of each word given the previous words"

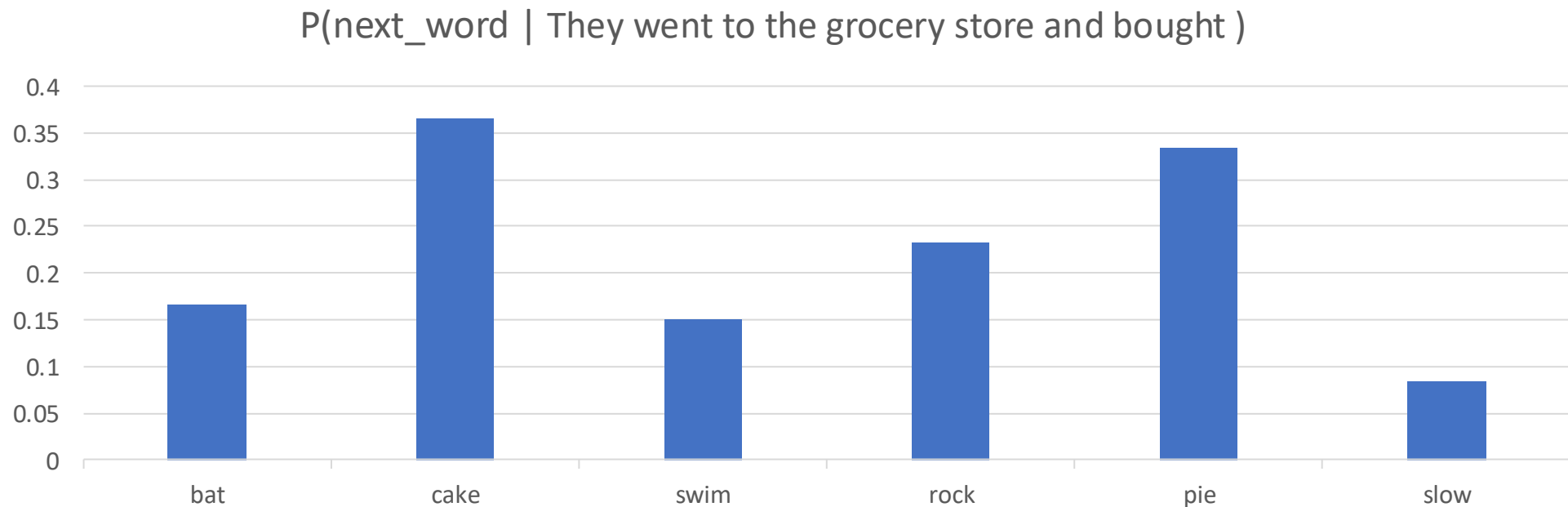
This is an application of the **chain rule for probabilities**

But first, how do we represent sentence?

Language models: the math

At each step, we look at a probability distribution for what the *next* word might be.

They went to the grocery store and bought ..



Natural language: tokenization

“They went to the grocery store and bought bread, peanut butter, and jam.”



[“they”, “went”, “to”, “the”,
“grocery”, “store”, “and”,
“bought”, “bread”, “peanut”,
“butter”, “and”, “jam”]

Natural language: tokenization

“They went to the grocery store and bought bread, peanut butter, and jam.”

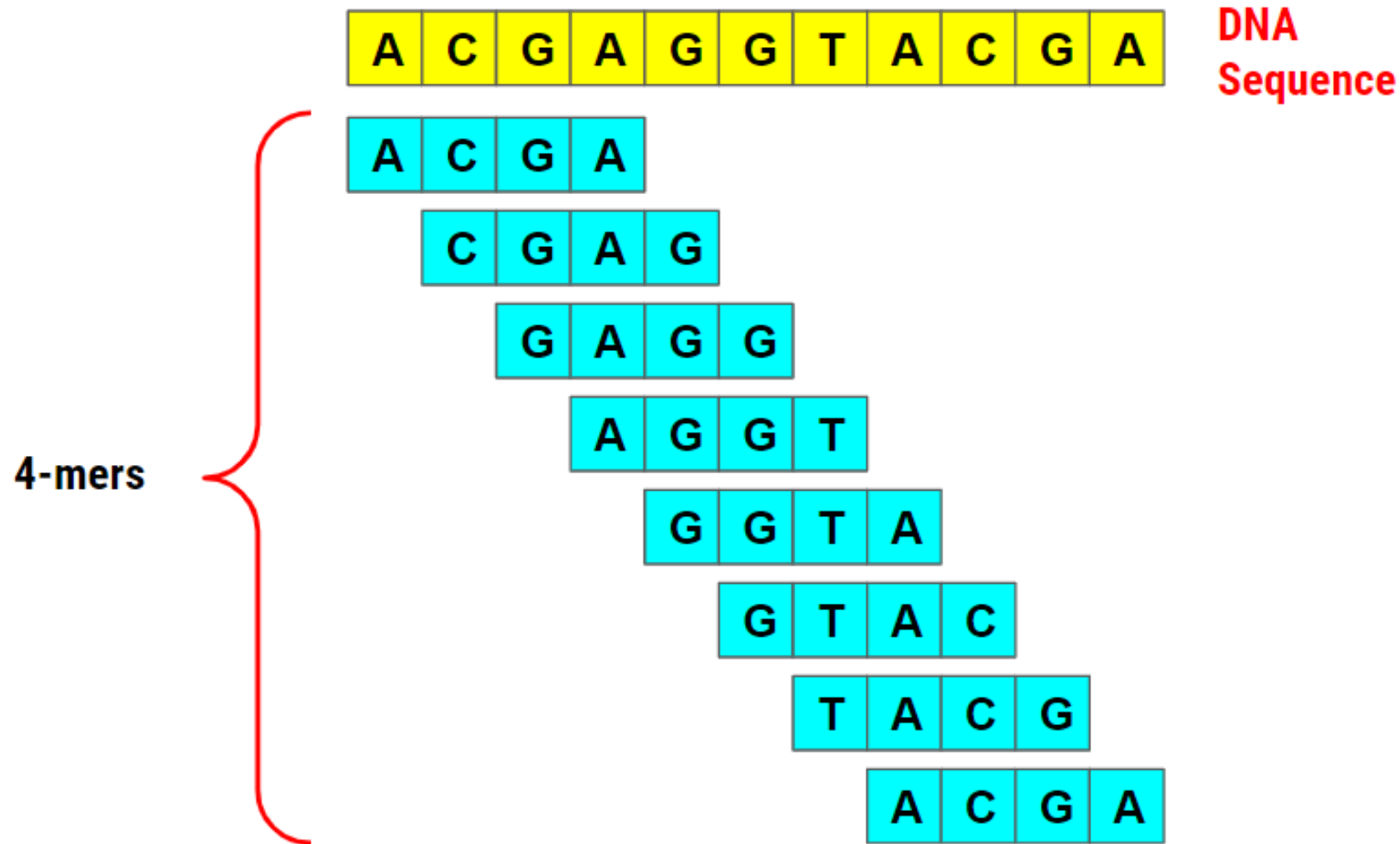
- Consistent casing
- Strip punctuation
- One word is one token
- Split on spaces

[“they”, “went”, “to”, “the”,
“grocery”, “store”, “and”,
“bought”, “bread”, “peanut”,
“butter”, “and”, “jam”]

Aside: Tokenization itself can be challenging...

- A lot easier in English than other languages (e.g. Chinese)
 - Chinese is character-based; words & phrases have different character lengths
 - No spaces

Speaking of complex tokenization: DNA (k-mer based)



Any questions?

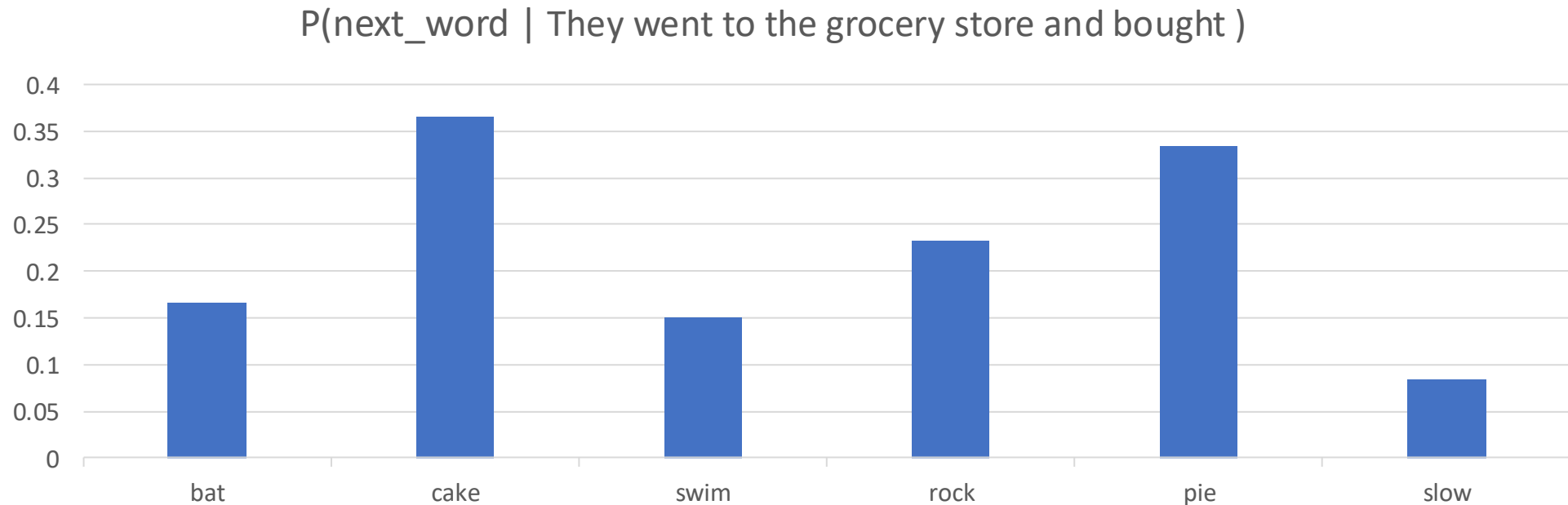


Language models: the math

How do we know which words to calculate probabilities for?

At each step, we look at a probability distribution for what the *next* word might be.

They went to the grocery store and bought ..



Vocabularies: Defining a finite set of words

Vocabularies: the set of all words “known” to the model

Why?

- We need a finite set of words in order to define a discrete distribution over it.

How?

- Choose a hyperparameter **vocab_size** for how many words the model should know
- Keep only the **vocab_size** with most frequent words – replace everything else with “**UNK**”

Vocabularies: how

- Original sentence:

- *“They galloped to the Ratty for dinner, and ate exactly seventy-three waffle fries and chocolate peamilk.”*

Vocabularies: how

- Original sentence:

- *“They galloped to the Ratty for dinner, and ate exactly seventy-three waffle fries and chocolate peamilk.”*

- Tokenized:

- [“they”, “galloped”, “to”, “the”, “ratty”, “for”, “dinner”, “and”, “ate”, “exactly”, “seventy-three”, “waffle”, “fries”, “and”, “chocolate”, “peamilk”]

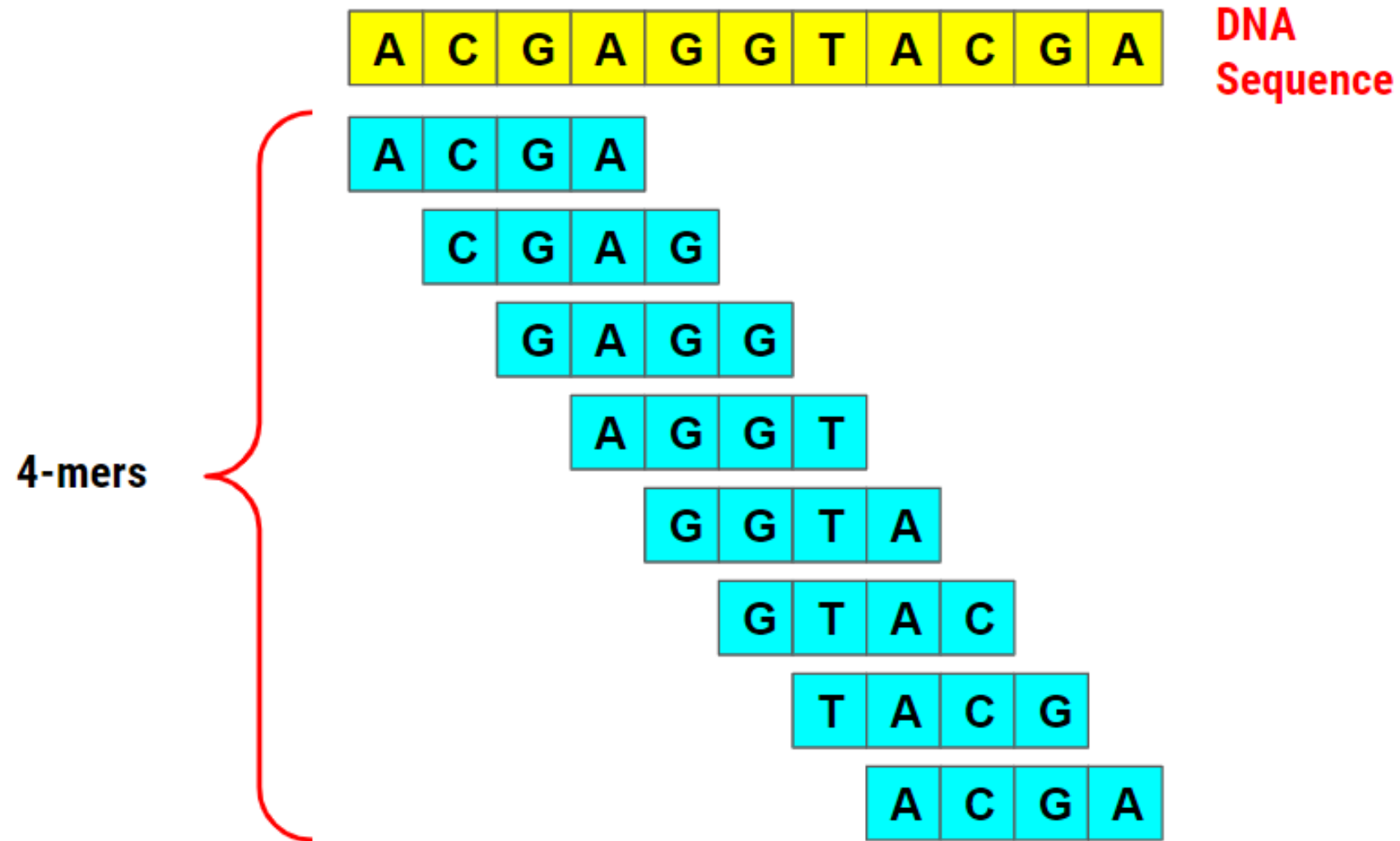
Vocabularies: how

- Original sentence:
 - *“They galloped to the Ratty for dinner, and ate exactly seventy-three waffle fries and chocolate peamilk.”*
- Tokenized:
 - [“they”, “galloped”, “to”, “the”, “ratty”, “for”, “dinner”, “and”, “ate”, “exactly”, “seventy-three”, “waffle”, “fries”, “and”, “chocolate”, “peamilk”]

Vocabularies: how

- Original sentence:
 - *“They galloped to the Ratty for dinner, and ate exactly seventy-three waffle fries and chocolate peamilk.”*
- Tokenized:
 - [“they”, “galloped”, “to”, “the”, “ratty”, “for”, “dinner”, “and”, “ate”, “exactly”, “seventy-three”, “waffle”, “fries”, “and”, “chocolate”, “peamilk”]
- UNKed:
 - [“they”, “UNK”, “to”, “the”, “UNK”, “for”, “dinner”, “and”, “ate”, “exactly”, “UNK”, “waffle”, “fries”, “and”, “chocolate”, “UNK”]

How would you define vocabulary for DNA sequence?

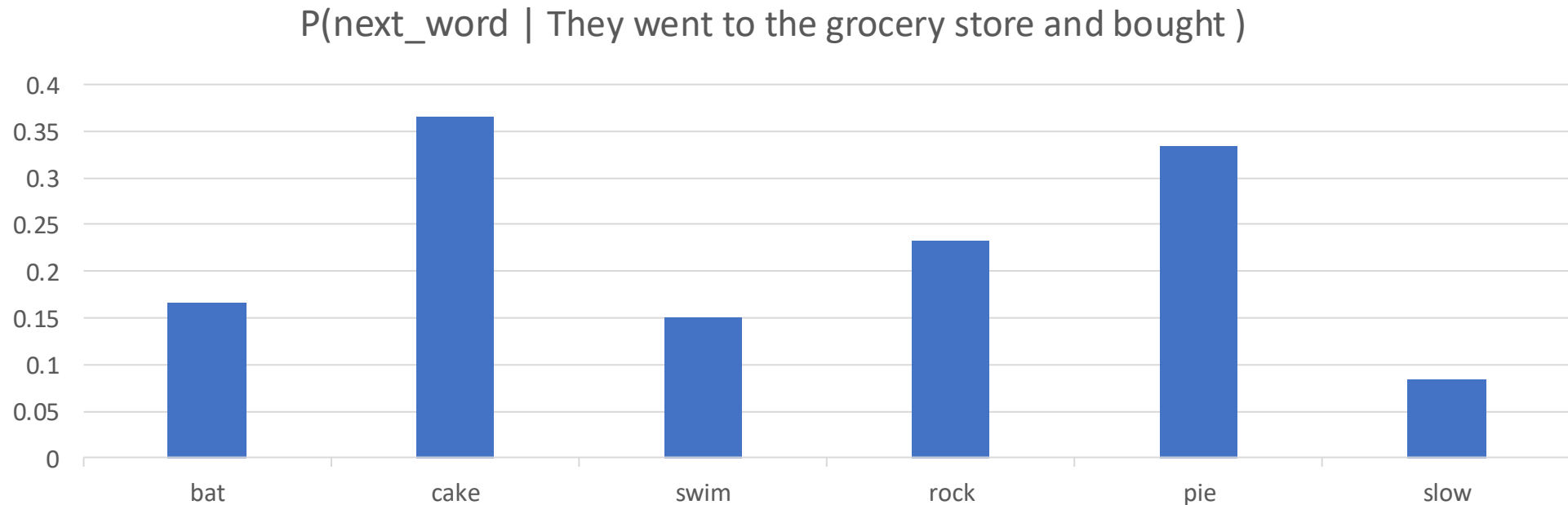


Language models: the math

How to calculate the probability for words in our vocabulary?

At each step, we look at a probability distribution for what the *next* word might be.

They went to the grocery store and bought ..



LM implementation: counting

- Goal: predict next word given a preceding sequence

- $P(\mathbf{word}_n | word_1, word_2, \dots word_{n-1}) = \frac{Count(word_1, word_2, \dots word_{n-1}, \mathbf{word}_n)}{Count(word_1, word_2, \dots word_{n-1})}$

LM implementation: counting

- Goal: predict next word given a preceding sequence

- $P(\mathbf{word}_n | word_1, word_2, \dots word_{n-1}) = \frac{Count(word_1, word_2, \dots word_{n-1}, \mathbf{word}_n)}{Count(word_1, word_2, \dots word_{n-1})}$

- Example task: predict the next word

- *he danced*

LM implementation: counting

- Goal: predict next word given a preceding sequence

- $P(\mathbf{word}_n | word_1, word_2, \dots word_{n-1}) = \frac{Count(word_1, word_2, \dots word_{n-1}, \mathbf{word}_n)}{Count(word_1, word_2, \dots word_{n-1})}$

- Example task: predict the next word

- *he danced*

- Strategy: iterate through all words in vocabulary, and calculate

- $$\frac{Count(\textit{he danced} \langle \textit{word} \rangle)}{Count(\textit{he danced})}$$
 for each word

LM implementation: counting

- Our training sentences were:

- “She danced happily”
- “They sang beautifully”
- “He danced energetically”
- “He sang happily”
- “She danced gracefully”

- “He danced _ _ _”

- “He danced **happily**”

Has 0 probability

$$\frac{\text{Count}(\text{he danced } < \text{word} >)}{\text{Count}(\text{he danced})}$$

Why doesn't this work?

This strategy depends on having instances of sentence prefixes.

LM implementation: N-gram counting

Improvement: **N-gram** model – only look at **N** words at a time

LM implementation: N-gram counting

Improvement: **N-gram** model – only look at **N** words at a time
(in this case, **bigrams** look at **2** words at a time)

- *“She danced happily”*
- *“They sang beautifully”*
- *“He danced energetically”*
- *“He sang happily”*
- *“She danced gracefully”*

LM implementation: N-gram counting

Improvement: **N-gram** model – only look at **N** words at a time
(in this case, **bigrams** look at **2** words at a time)

- “*danced happily*”
- “*sang beautifully*”
- “*danced energetically*”
- “*sang happily*”
- “*danced gracefully*”

“*He danced happily*” now has 1/3 probability!

But what if the answer was “*He danced beautifully*” ?

LM implementation

Problem: it's impossible for the training set to have *every possible valid* sequence of words!

Let's try to learn a better **numerical** representation

What is the simplest thing you can think of?

LM implementation: Simple approach

- “She danced happily”
- “They sang beautifully”
- “He danced energetically”
- “He sang happily”
- “She danced gracefully”

vocab_sz

Any potential issues with this?

“They danced happily”

⋮	⋮	⋮
0	0	0
1	0	0
0	1	0
0	0	0
0	0	1
⋮	⋮	⋮

LM implementation

Problem: it's impossible for the training set to have *every possible valid* sequence of words!

Can we learn a better numerical representation **which associates related words with one another?**

Language embedding

Embedding matrix

vocab_sz {

⋮	
<i>they</i>	2 0 1 3 0 4
<i>danced</i>	0 1 1 0 2 1
<i>sang</i>	0 0 2 0 1 3
<i>happily</i>	0 1 1 1 0 2
<i>gleefully</i>	4 0 0 1 1 0
⋮	

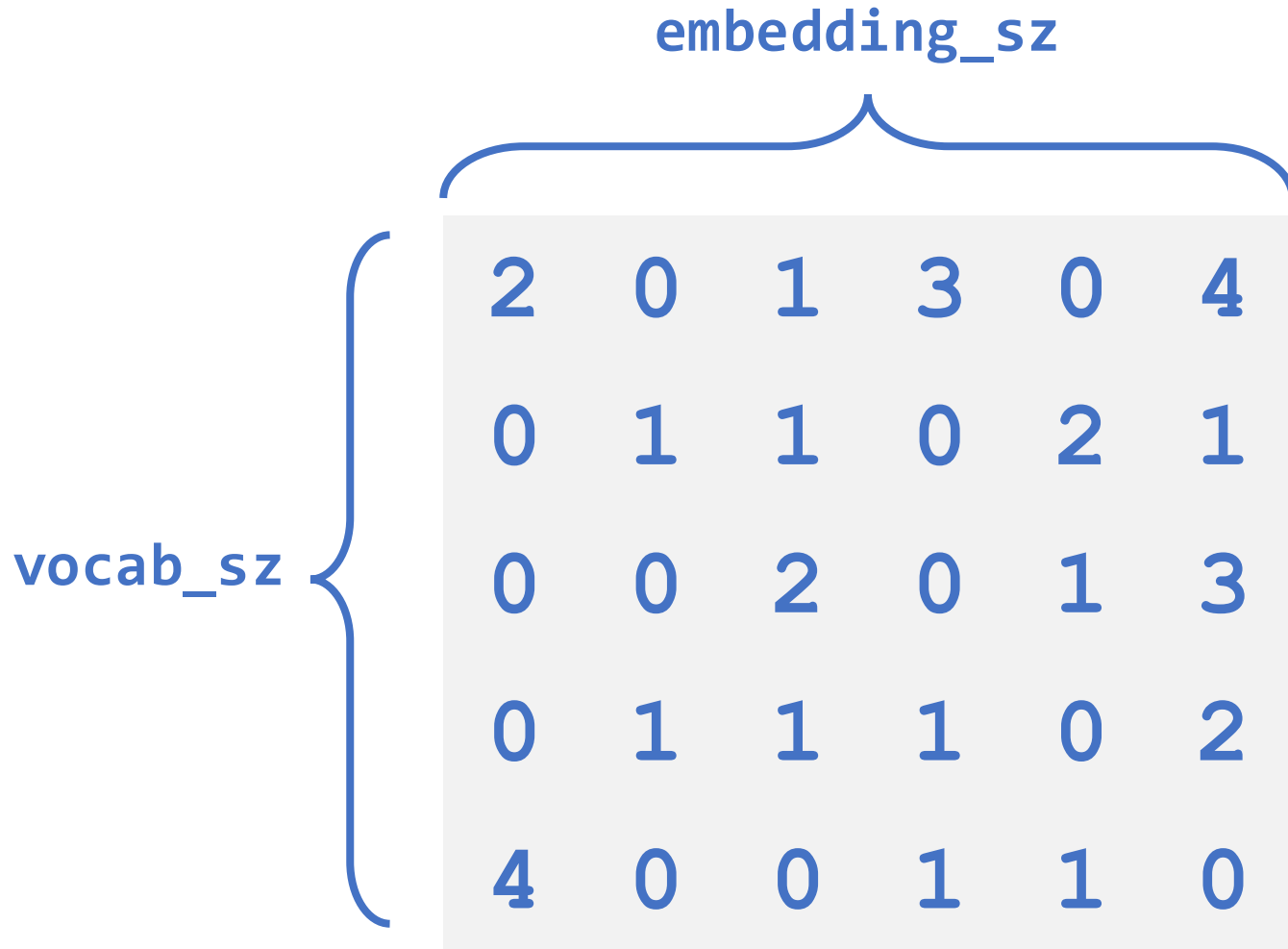
Embedding matrix

Any questions?



vocab_sz {		⋮						
		<i>they</i>	2	0	1	3	0	4
		<i>danced</i>	0	1	1	0	2	1
		<i>sang</i>	0	0	2	0	1	3
		<i>happily</i>	0	1	1	1	0	2
		<i>gleefully</i>	4	0	0	1	1	0
		⋮						

Embedding matrix

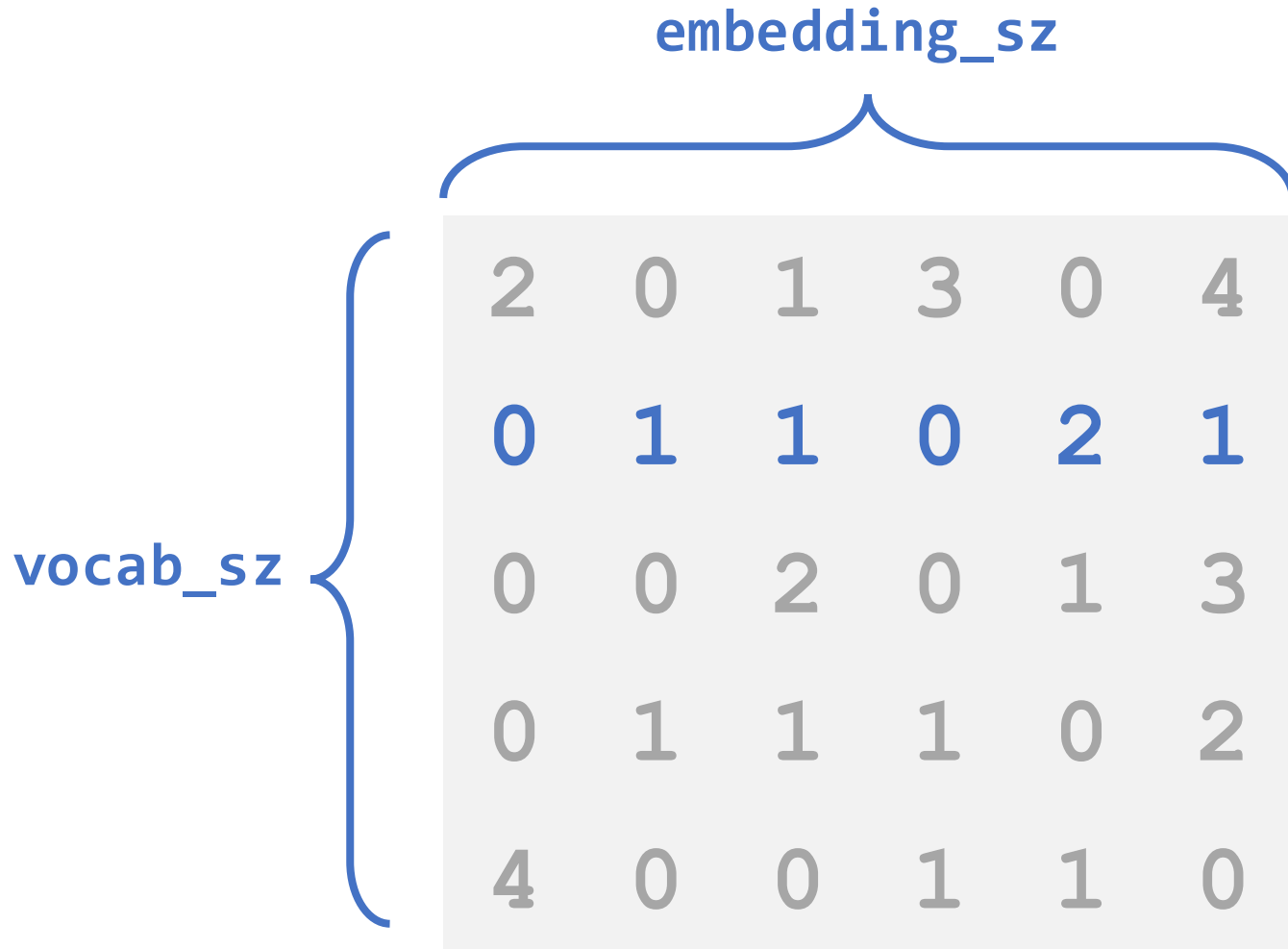


The diagram shows a 5x6 matrix of integers. A horizontal bracket above the matrix is labeled 'embedding_sz' and spans all six columns. A vertical bracket to the left of the matrix is labeled 'vocab_sz' and spans all five rows.

2	0	1	3	0	4
0	1	1	0	2	1
0	0	2	0	1	3
0	1	1	1	0	2
4	0	0	1	1	0

- 2d matrix: **vocab_sz**
x embedding_sz

Embedding matrix



The diagram shows a 6x6 matrix representing an embedding matrix. A blue bracket on the left side, labeled `vocab_sz`, spans all six rows. A blue bracket on the top side, labeled `embedding_sz`, spans all six columns. The matrix contains numerical values, with the second row highlighted in blue.

2	0	1	3	0	4
0	1	1	0	2	1
0	0	2	0	1	3
0	1	1	1	0	2
4	0	0	1	1	0

- 2d matrix: **vocab_sz** x **embedding_sz**
- each word corresponds to an index, or word ID – hence the **vocab_sz** dimension

Embedding matrix

How to build this embedding matrix?

embedding_sz

vocab_sz

2	0	1	3	0	4
0	1	1	0	2	1
0	0	2	0	1	3
0	1	1	1	0	2
4	0	0	1	1	0

- 2d matrix: **vocab_sz** x **embedding_sz**
- each word corresponds to an index, or word ID – hence the **vocab_sz** dimension
- **embedding_sz** is a hyperparameter

LM implementation: deep learning

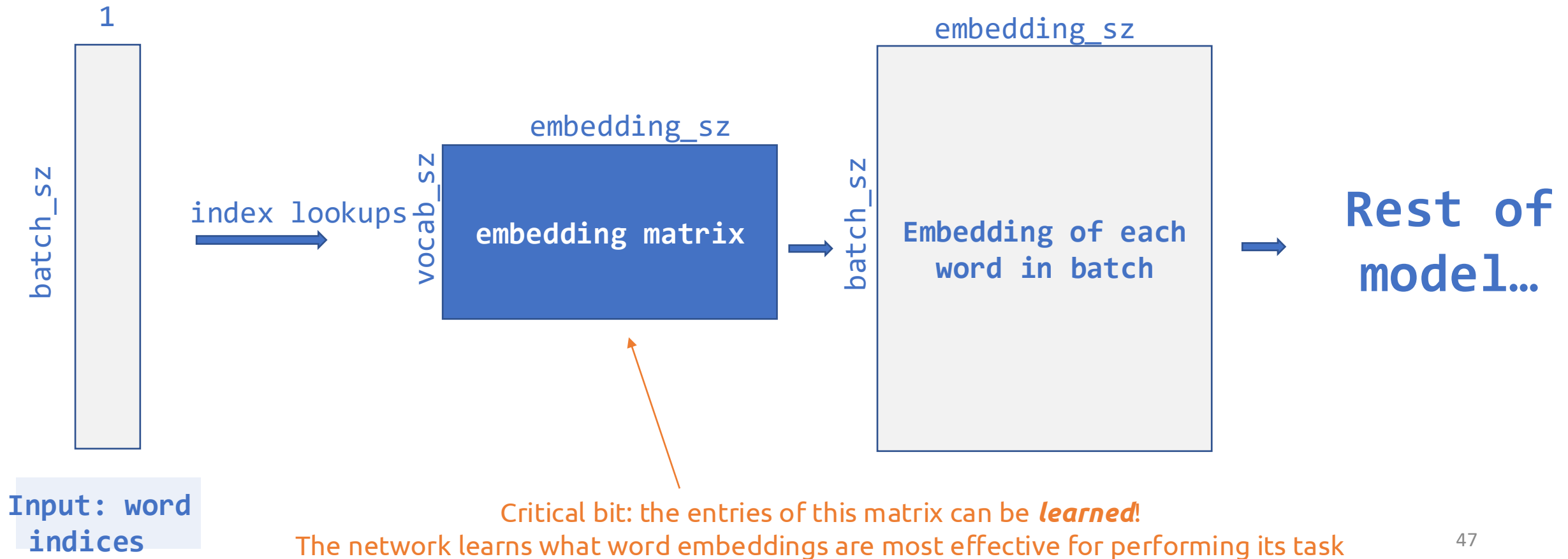
Deep learning helps solve this! **How?**

We can learn an ***embedding matrix*** that associates *related* words with one another for solving a prediction task.

Using the Embedding Matrix in a Network

If you want to input a [batch of] words into a neural net, this is how:

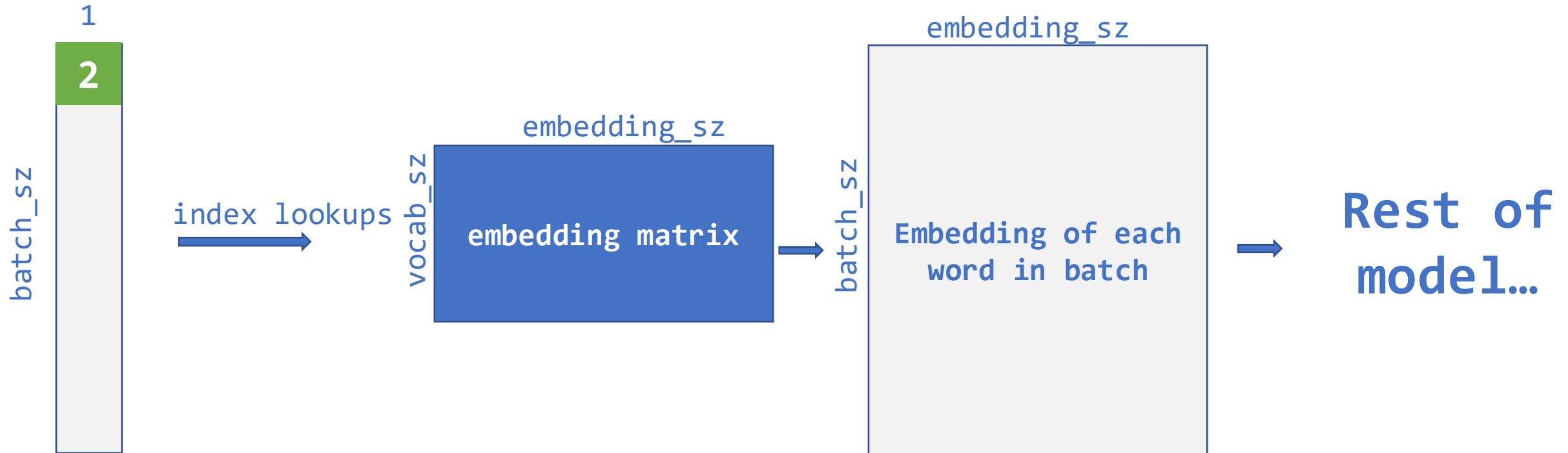
they, danced, happily



Using the Embedding Matrix in a Network

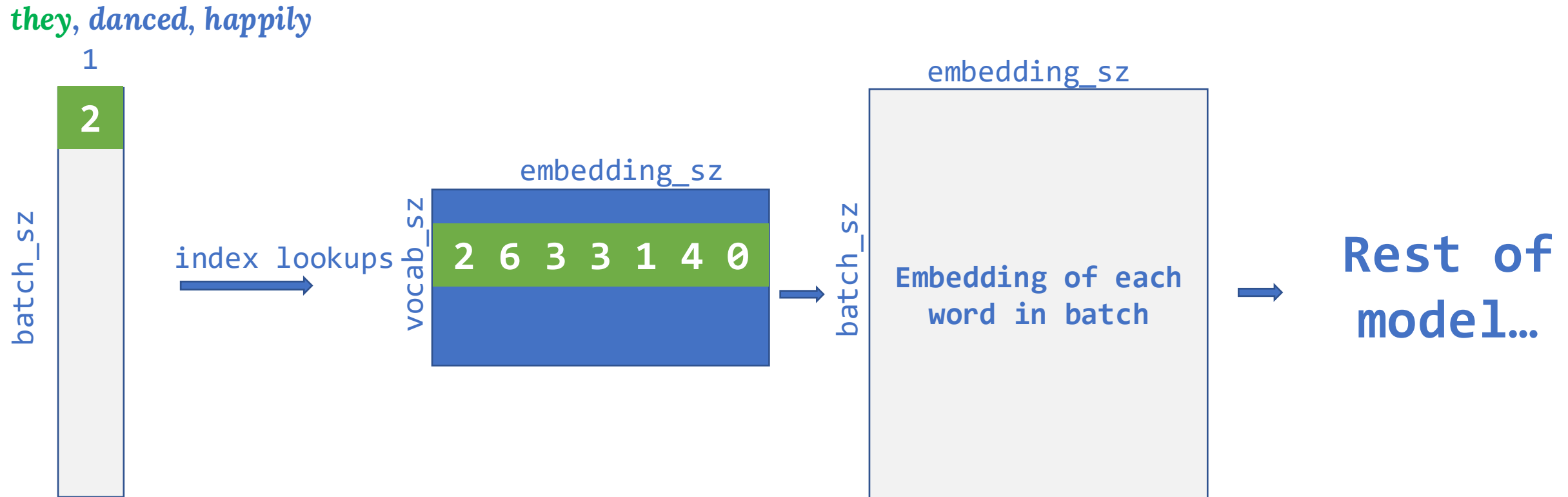
Let's look at the 0th word in this batch; its ID in the vocab is 2.

they, danced, happily



Using the Embedding Matrix in a Network

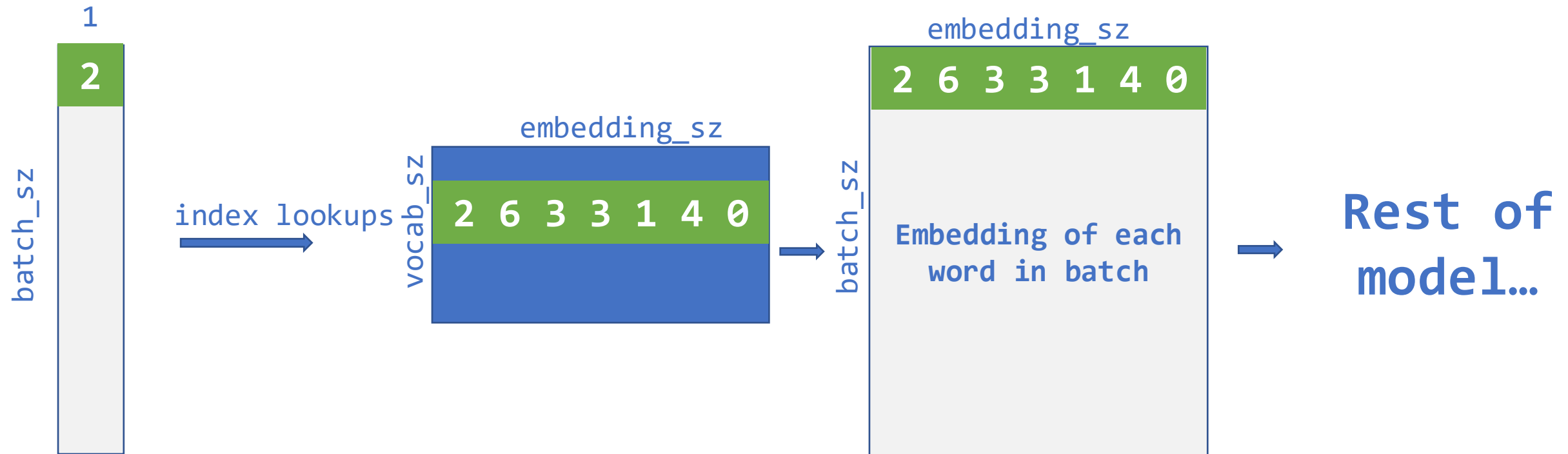
So we look at row 2 of the embedding matrix.



Using the Embedding Matrix in a Network

We can then pull out this embedding so we can use it in the rest of the model!

they, danced, happily

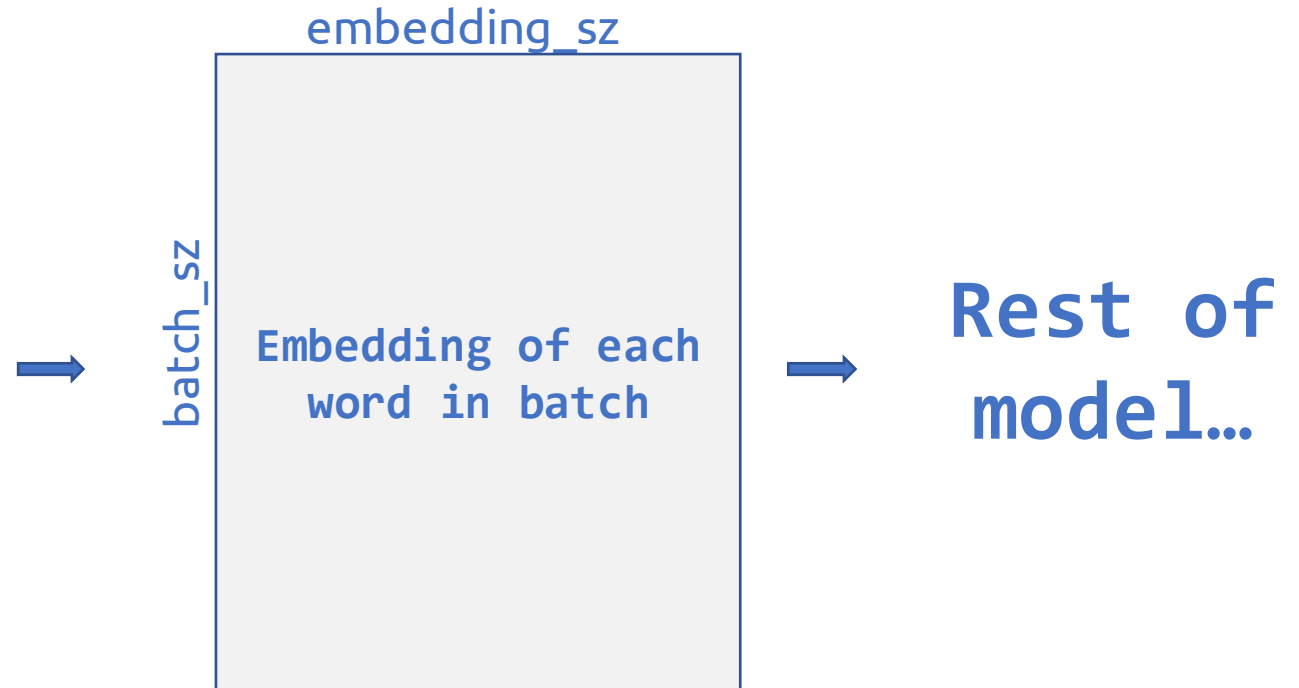


Using the Embedding Matrix in a Network

In Pytorch, we can use

```
torch.nn.functional.embedding  
(input, weight)
```

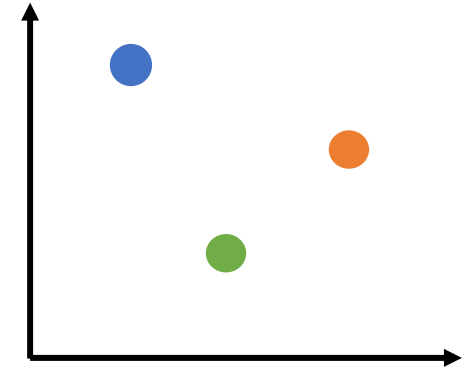
which takes in an embedding matrix and a list of indices, and returns the embedding corresponding to each index.



What does the embedding matrix represent?

- Each row in the matrix can be viewed as a vector in vector space

Example 2-D
vector space:



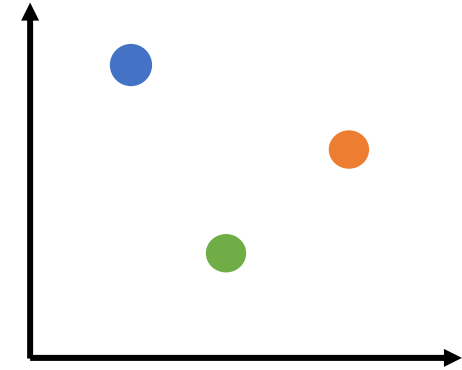
Vocab size: 3
Embed size: 2

1	3
2	1
3	2

What does the embedding matrix represent?

- Each row in the matrix can be viewed as a vector in vector space
- “Embedding”: We’re **embedding** a non-Euclidian entity [a word] into Euclidian space

Example 2-D
vector space:



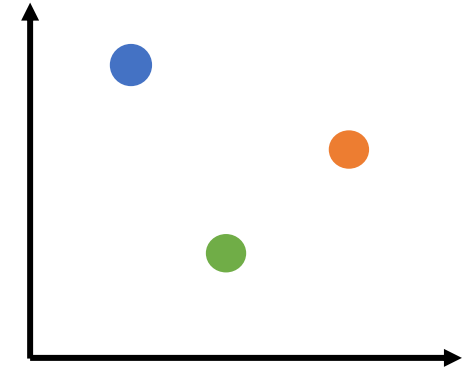
Vocab size: 3
Embed size: 2

1	3
2	1
3	2

What does the embedding matrix represent?

- Each row in the matrix can be viewed as a vector in vector space
- “Embedding”: We’re **embedding** a non-Euclidian entity [a word] into Euclidian space
- Each row represents the “embedding” for a single word

Example 2-D
vector space:



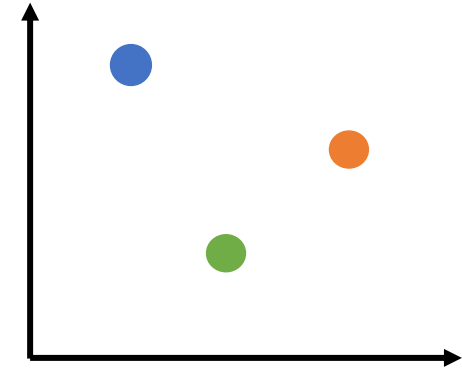
Vocab size: 3
Embed size: 2

1	3
2	1
3	2

What does the embedding matrix represent?

- Each row in the matrix can be viewed as a vector in vector space
- “Embedding”: We’re **embedding** a non-Euclidian entity [a word] into Euclidian space
- Each row represents the “embedding” for a single word
- This has pretty remarkable properties!

Example 2-D
vector space:

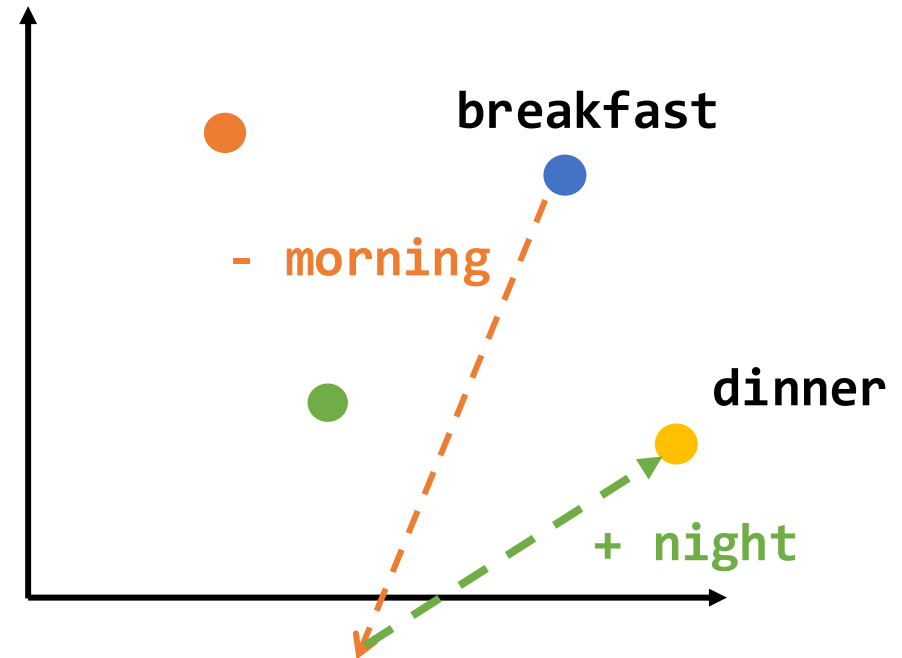
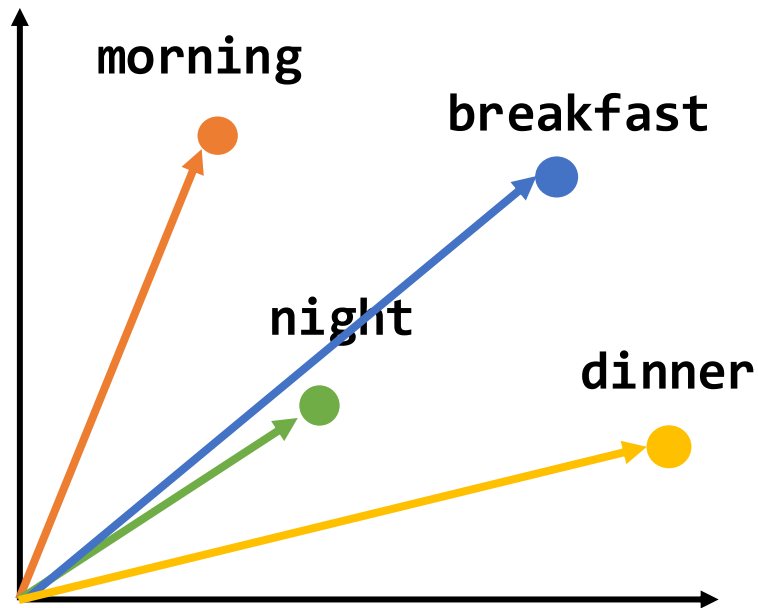


Vocab size: 3
Embed size: 2

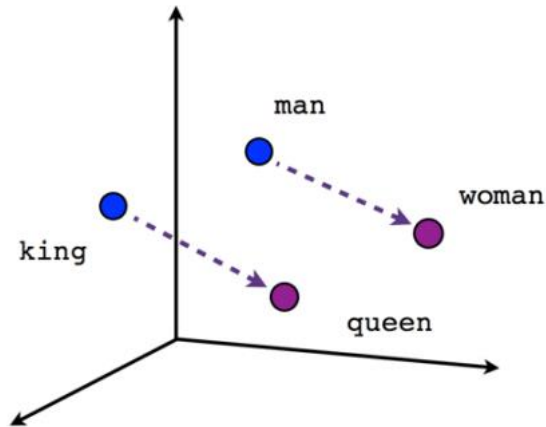
1	3
2	1
3	2

Vector arithmetic in the embedding matrix

Demo [here](#)



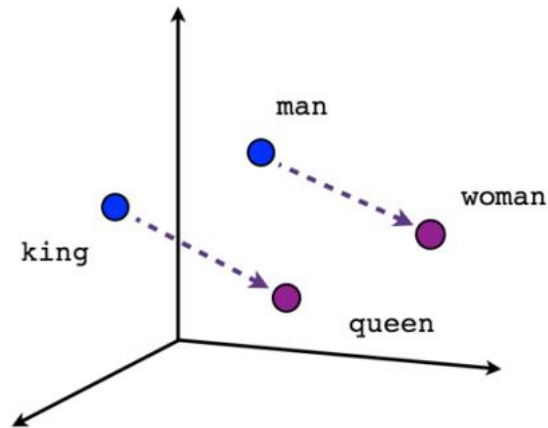
More 'semantic directions' in embedding space



Male-Female

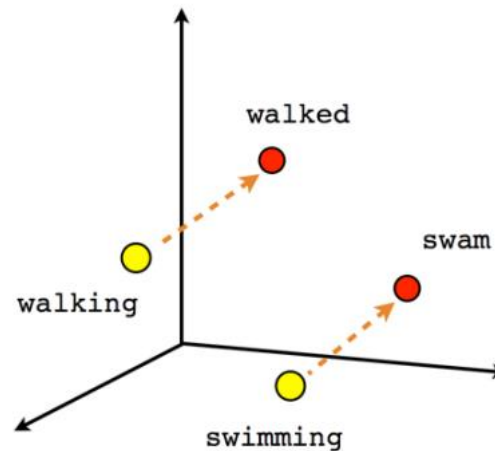
$$\begin{aligned} E(\text{queen}) - E(\text{king}) &\approx \\ E(\text{woman}) - E(\text{man}) \end{aligned}$$

More 'semantic directions' in embedding space



Male-Female

$$\begin{aligned} E(\text{queen}) - E(\text{king}) &\approx \\ E(\text{woman}) - E(\text{man}) \end{aligned}$$

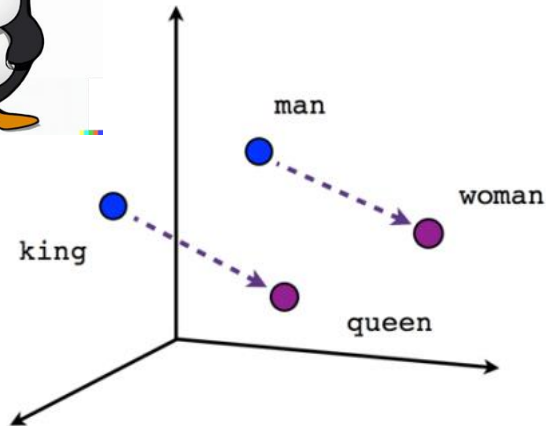


Verb tense

$$\begin{aligned} E(\text{walked}) - E(\text{walking}) &\approx \\ E(\text{swam}) - E(\text{swimming}) \end{aligned}$$

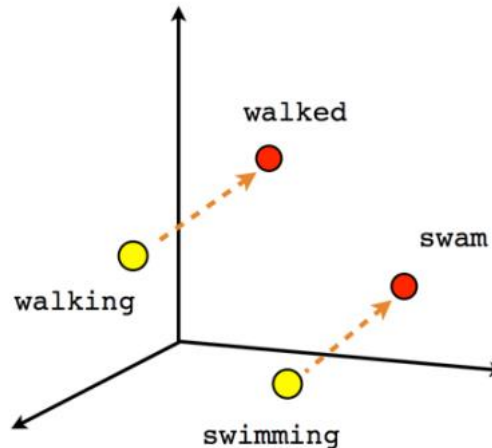
More 'semantic directions' in embedding space

Any questions?



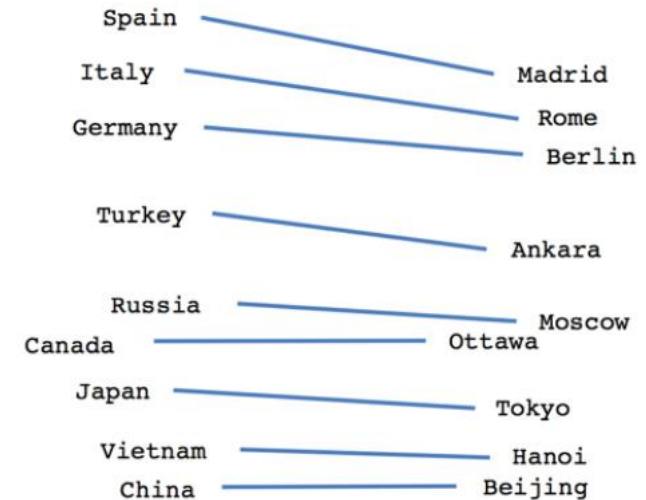
Male-Female

$$\begin{aligned} E(\text{queen}) - E(\text{king}) &\approx \\ E(\text{woman}) - E(\text{man}) \end{aligned}$$



Verb tense

$$\begin{aligned} E(\text{walked}) - E(\text{walking}) &\approx \\ E(\text{swam}) - E(\text{swimming}) \end{aligned}$$

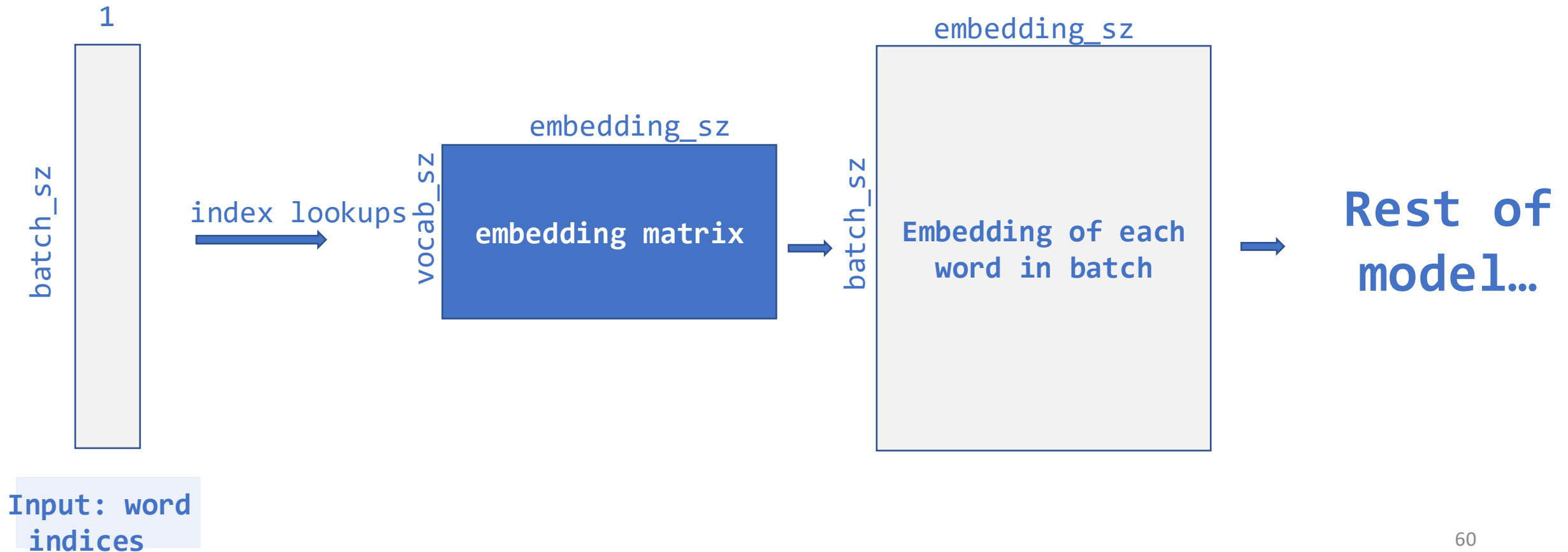


Country-Capital

$$\begin{aligned} E(\text{Spain}) - E(\text{Madrid}) &\approx \\ E(\text{Vietnam}) - E(\text{Hanoi}) \end{aligned}$$

Why do embedding matrices work like this?

When the language model is trained, it's incentivized to put words with similar context near each other in the embedding space.



Why do embedding matrices work like this?

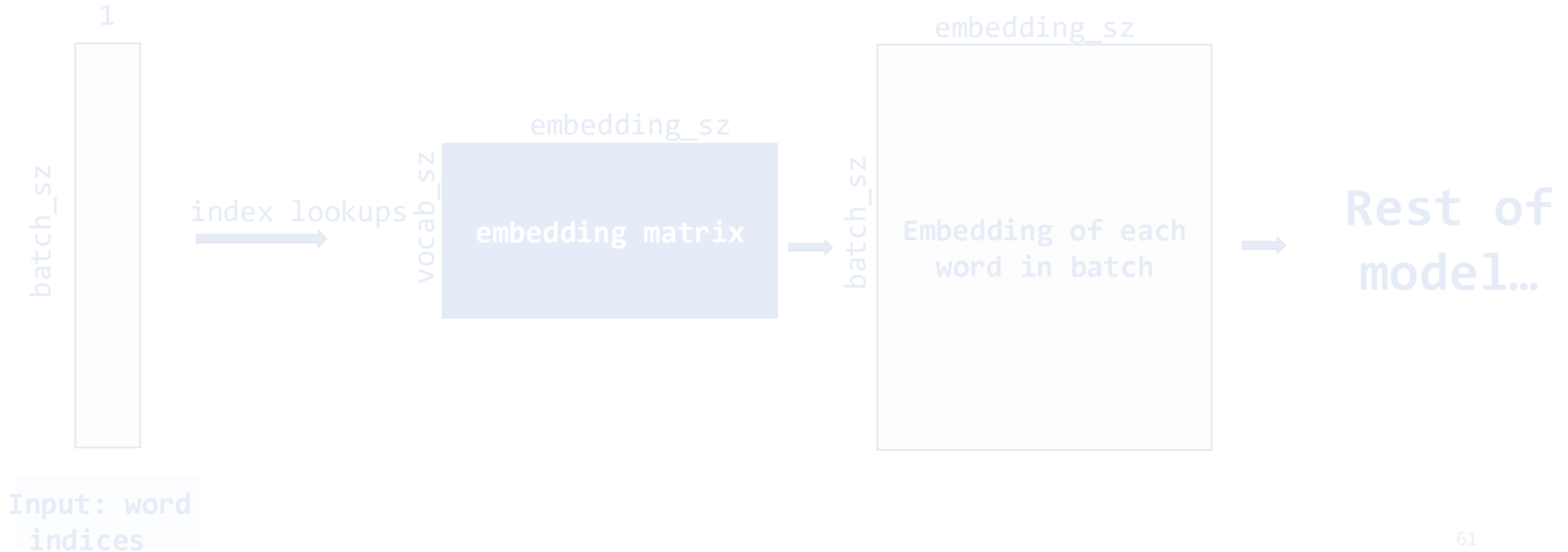
Let's say in the middle of training...

Then, the model sees a lot of "danced gleefully"

$P(\text{"happily"} | \text{"they danced"}) = \text{high}$

How do we increase $P(\text{"gleefully"} | \text{"they danced"})$?

$P(\text{"gleefully"} | \text{"they danced"}) = \text{low}$



Why do embedding matrices work like this?

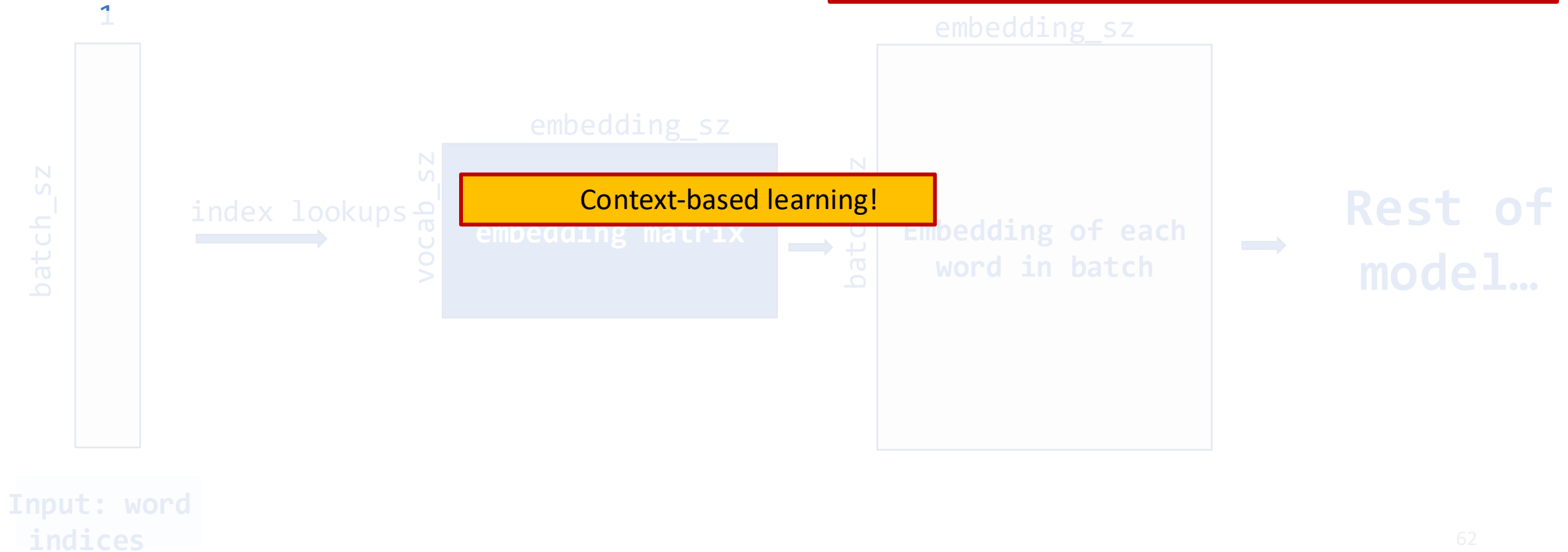
Let's say in the middle of training...

$P(\text{"happily"} | \text{"they danced"}) = \text{high}$

$P(\text{"gleefully"} | \text{"they danced"}) = \text{low}$

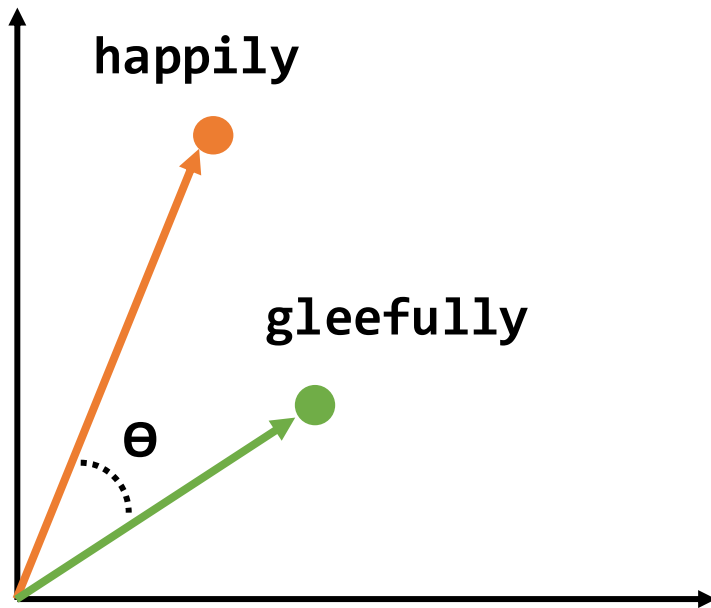
Since probability is calculated based on the embedding matrix...

Modify the embedding of **"gleefully"** so that it's similar to the embedding of **"happily"**!



Quantifying “similarity”

$$\text{cosine similarity} = \cos(\theta) = \frac{A \cdot B}{||A|| ||B||} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



$$\begin{aligned}\cos(0^\circ) &= 1 \\ \cos(90^\circ) &= -0.448 \\ \cos(180^\circ) &= -0.598\end{aligned}$$

Class activity: DNA2Vec

Paper: [dna2vec: Consistent vector representations of variable-length k-mers](#)

[Implementing DNA2Vec using Word2Vec model](#)

N-gram Language Models

Once upon a time...



“The dog barked loudly.”



“The cat meowed softly”

Data Preprocessing

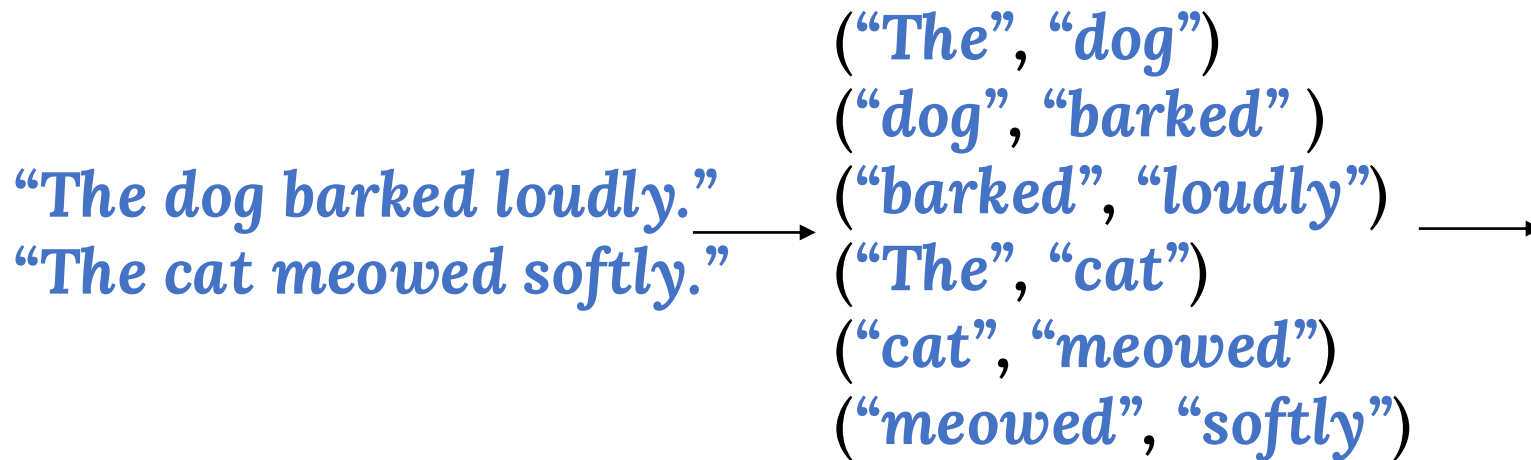
First, we extract all of the bigrams from the training corpus.

“The dog barked loudly.”
“The cat meowed softly.” → *(“The”, “dog”)*
(“dog”, “barked”)
(“barked”, “loudly”)
(“The”, “cat”)
(“cat”, “meowed”)
(“meowed”, “softly”)

Data Preprocessing

First, we extract all of the bigrams from the training corpus.

Create training batches by pairing up first and second words:



inputs	labels
<i>“The”</i>	<i>“dog”</i>
<i>“dog”</i>	<i>“barked”</i>
<i>“barked”</i>	<i>“loudly”</i>
<i>“The”</i>	<i>“cat”</i>
<i>“cat”</i>	<i>“meowed”</i>
<i>“meowed”</i>	<i>“softly”</i>

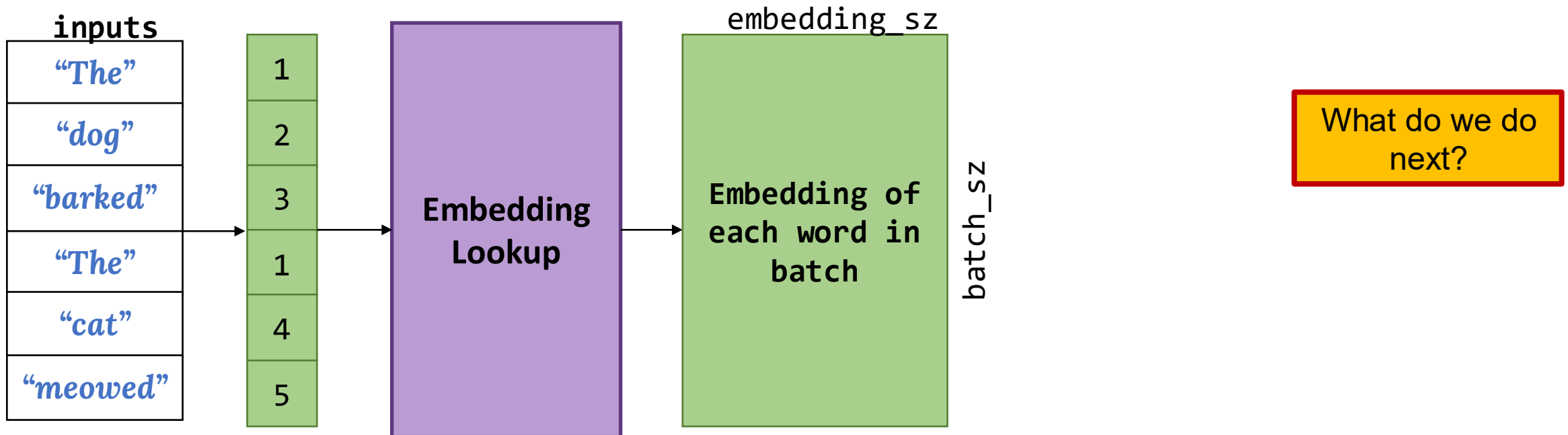
Bigram Language Model Architecture

We convert all words to their corresponding vocab indices.

inputs	
"The"	1
"dog"	2
"barked"	3
"The"	1
"cat"	4
"meowed"	5

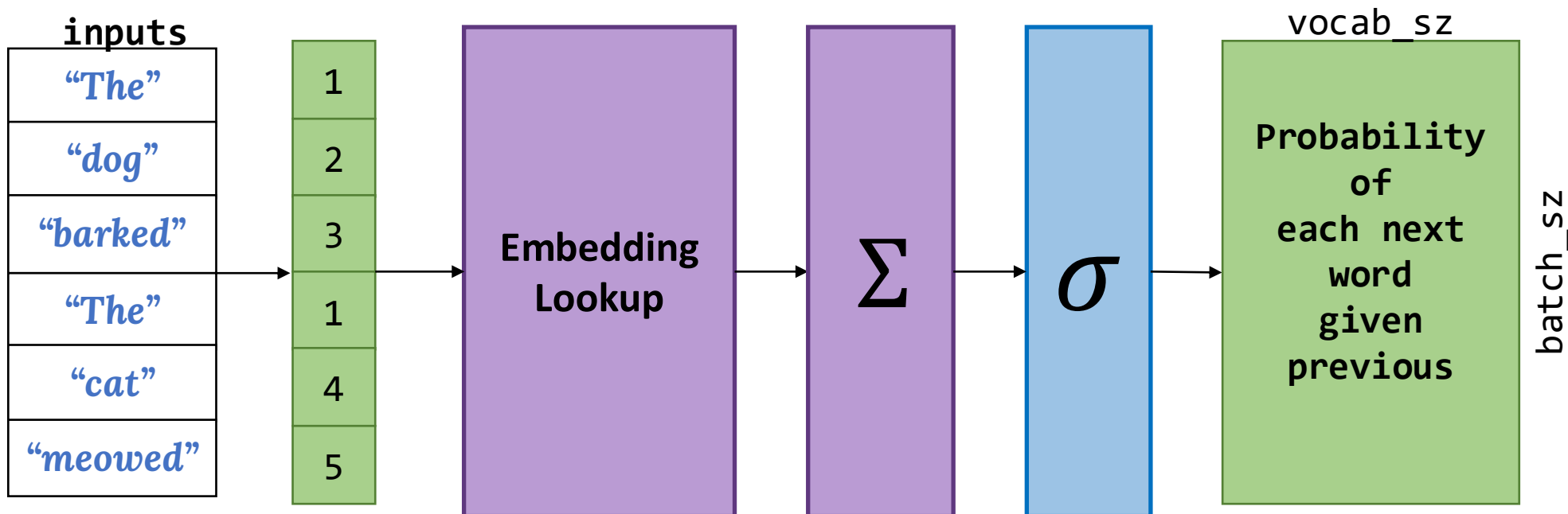
Bigram Language Model Architecture

Applying **lookup** to our entire batch gets the embedding for each word in the batch.



Bigram Language Model Architecture

We feed our batch of embeddings to a fully connected layer with softmax activation to get probability of each word in vocab.

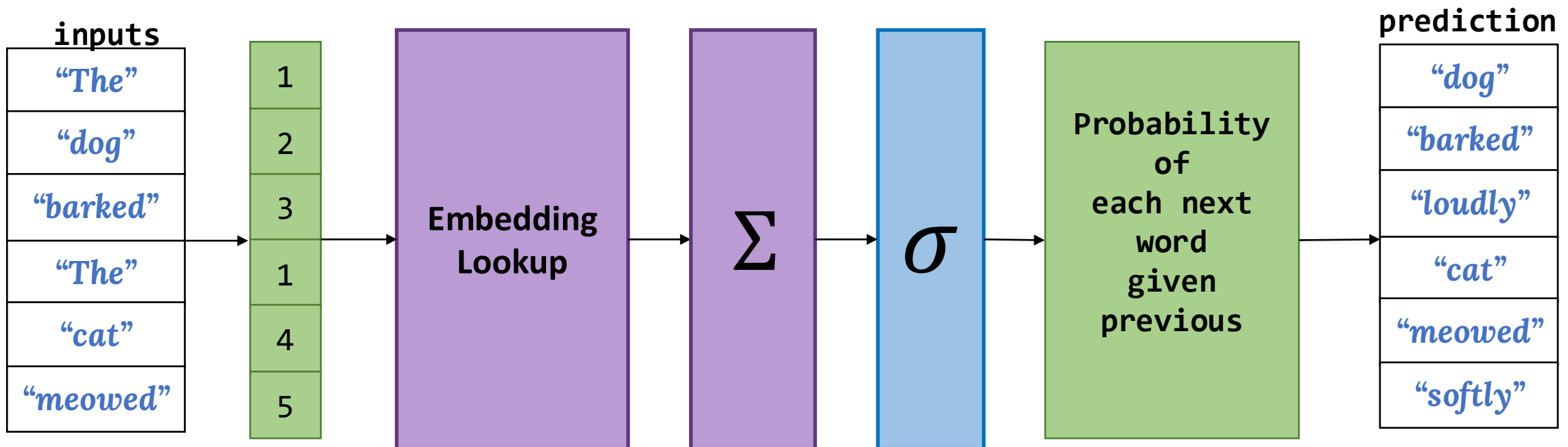


Bigram Language Model Architecture

Any questions?



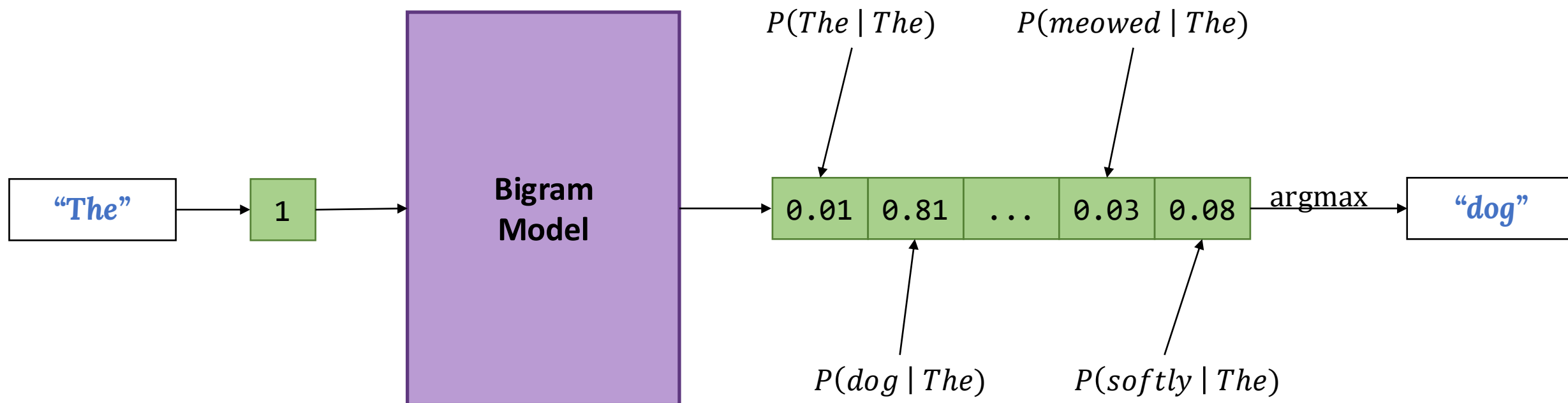
Finally, we choose the word with the max probability as our prediction.



Bigram Language Model Output

Should we be concerned about a large vocabulary making the probabilities small?

The output of our model gives us the probability of each word in our vocabulary appearing next, given the previous word.



Why might the bigram model not be sufficient?

Improving on the Bigram model

Why might the bigram model not be sufficient?

Consider slightly more *distant* sentence relationships:

“The dog was barking.”



“The cat was meowing.”



Improving on the Bigram model

Why might the bigram model not be sufficient?

Consider slightly more *distant* sentence relationships:

“The dog was barking.”


“The cat was meowing.”


We want to capture *context* farther than the immediately preceding word.

Using the bigram model, we would need to predict “*barking*” and “*meowing*” based only on the word “*was*”.

Can we do better?

Trigram Language Model

“The dog was barking.”
“The cat was meowing.”

For the trigram model, we treat the first two words of each trigram as the input, and the third word as the target.

(“The”, “dog”, “was”)
(“dog”, “was”, “barking”)
(“The”, “cat”, “was”)
(“cat”, “was”, “meowing”)

inputs		labels
<i>“The”</i>	<i>“dog”</i>	<i>“was”</i>
<i>“dog”</i>	<i>“was”</i>	<i>“barking”</i>
<i>“The”</i>	<i>“cat”</i>	<i>“was”</i>
<i>“cat”</i>	<i>“was”</i>	<i>“meowing”</i>

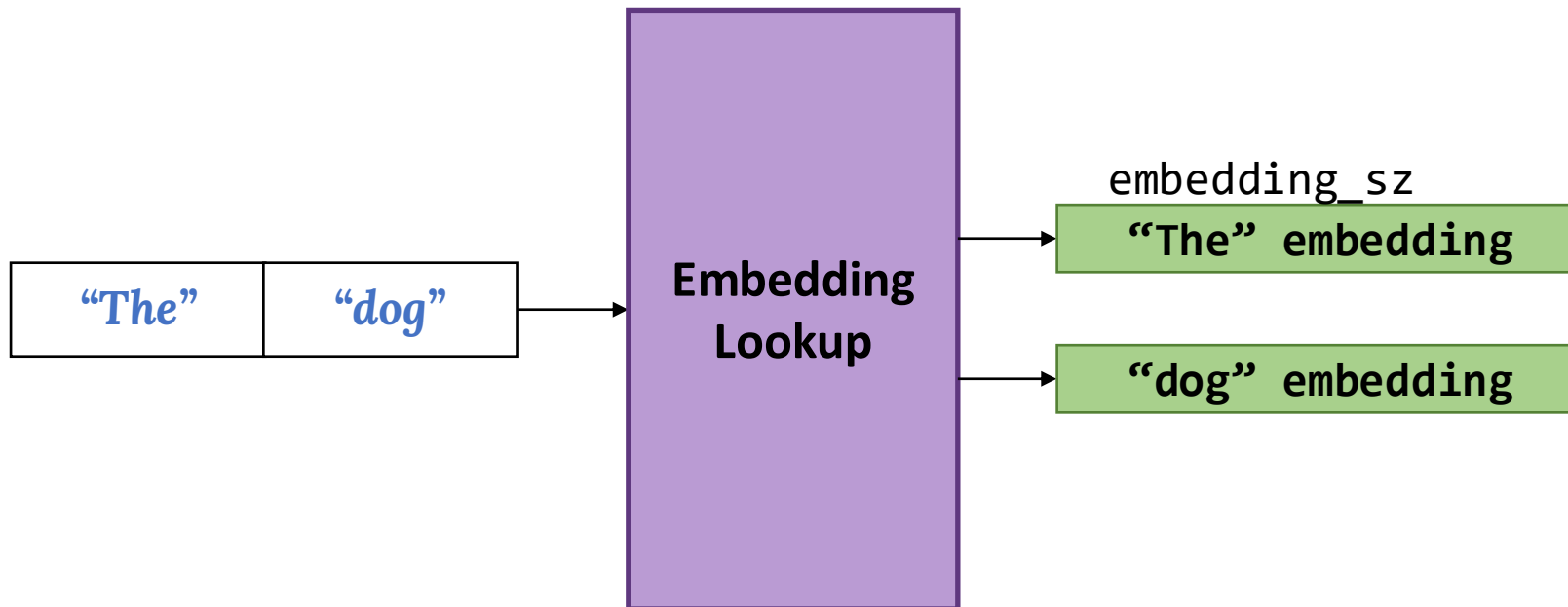
Trigram Language Model Input

Now our network input is two words...

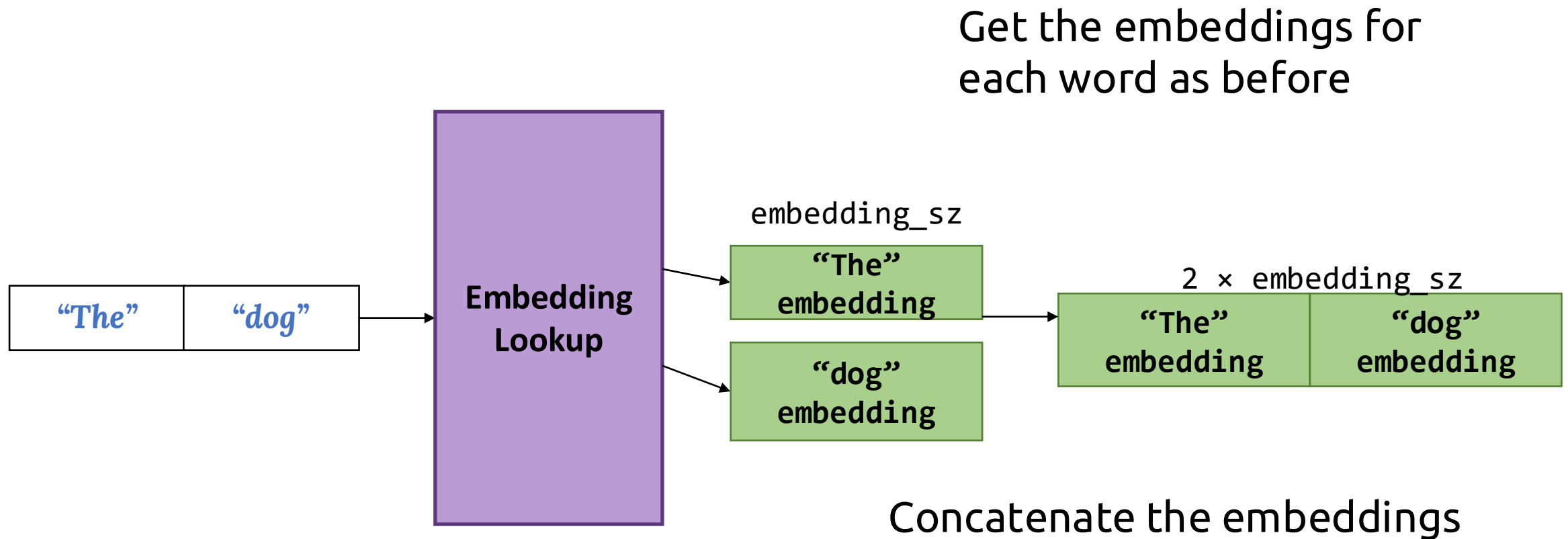
...how do we turn these into a tensor to feed into our network?

Handling Multi-Word Input

Get the embeddings for each word as before



Handling Multi-Word Input

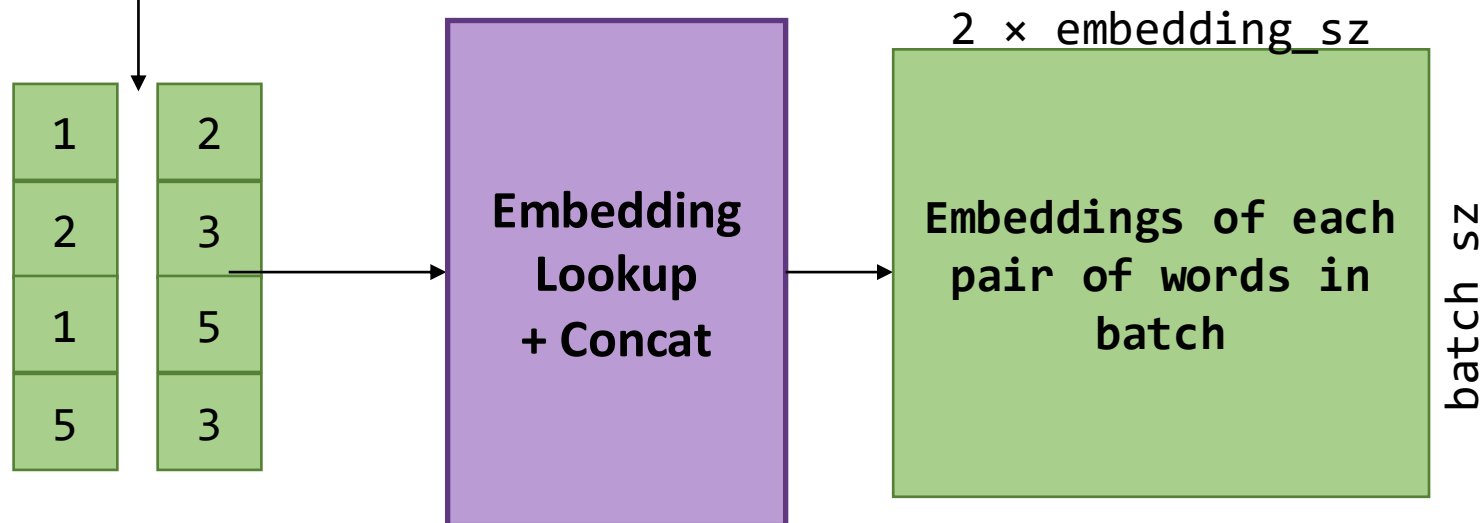


Complete Trigram Language Model

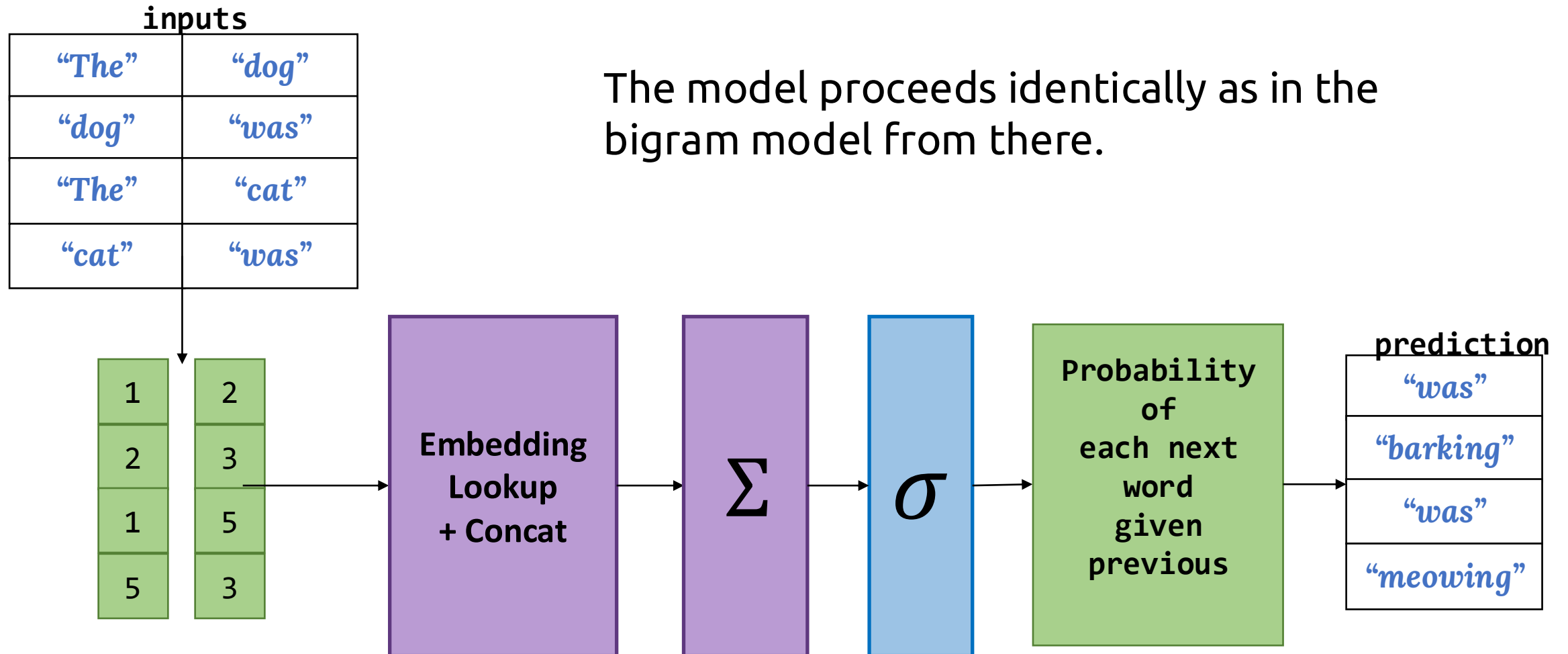
inputs

<i>"The"</i>	<i>"dog"</i>
<i>"dog"</i>	<i>"was"</i>
<i>"The"</i>	<i>"cat"</i>
<i>"cat"</i>	<i>"was"</i>

First we get the concatenated embeddings of the word pairs.



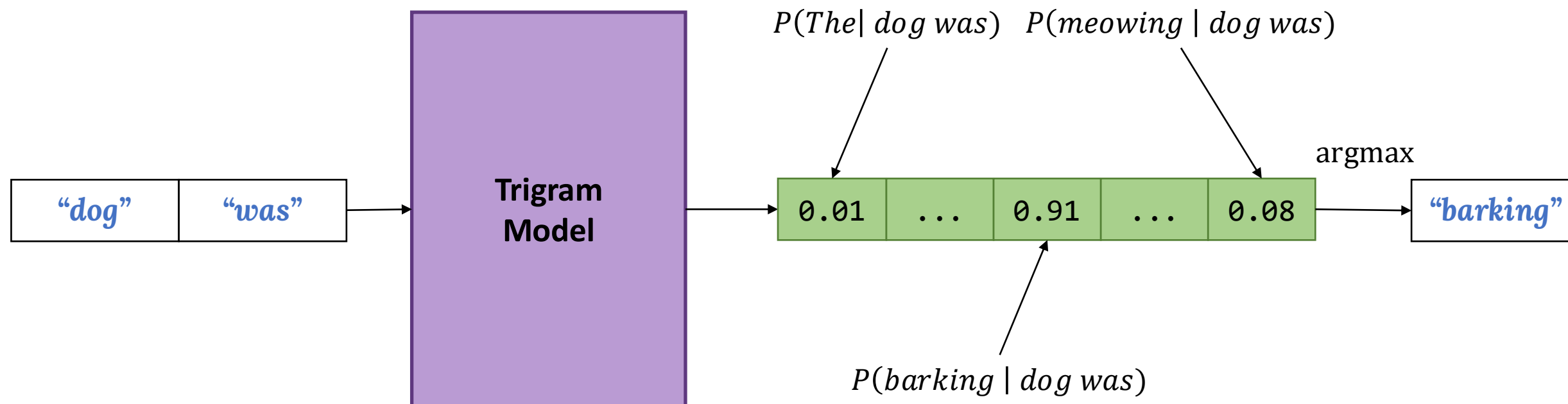
Complete Trigram Language Model





Trigram Language Model Output

In the trigram version, the probabilities are now conditioned on two previous words rather than just one.

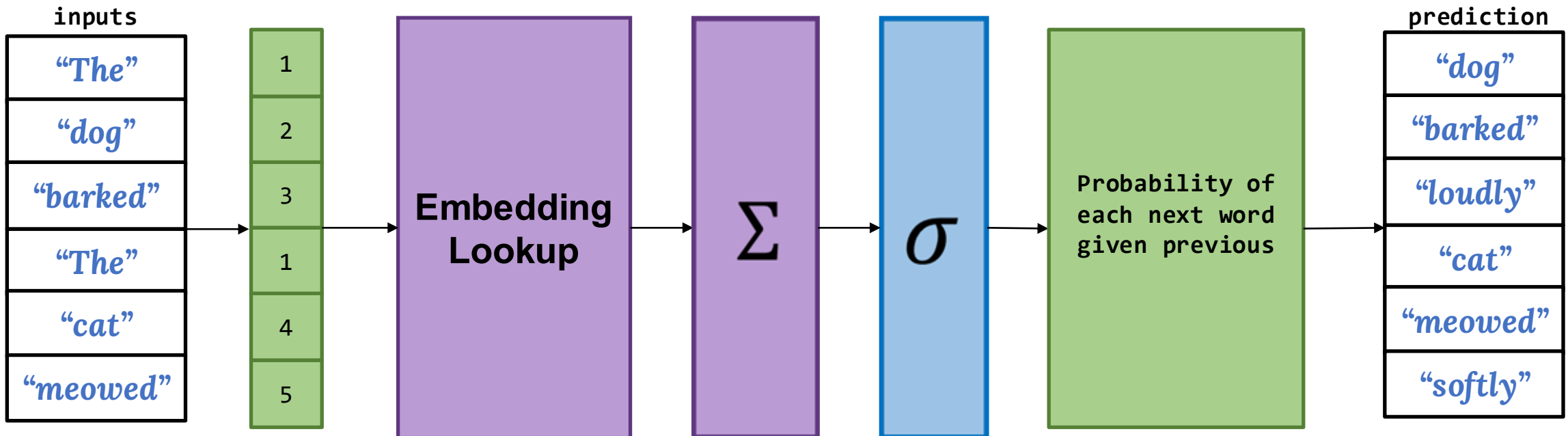


Limitations of the N-gram model

What problems do we run into using Feed Forward N-gram models?

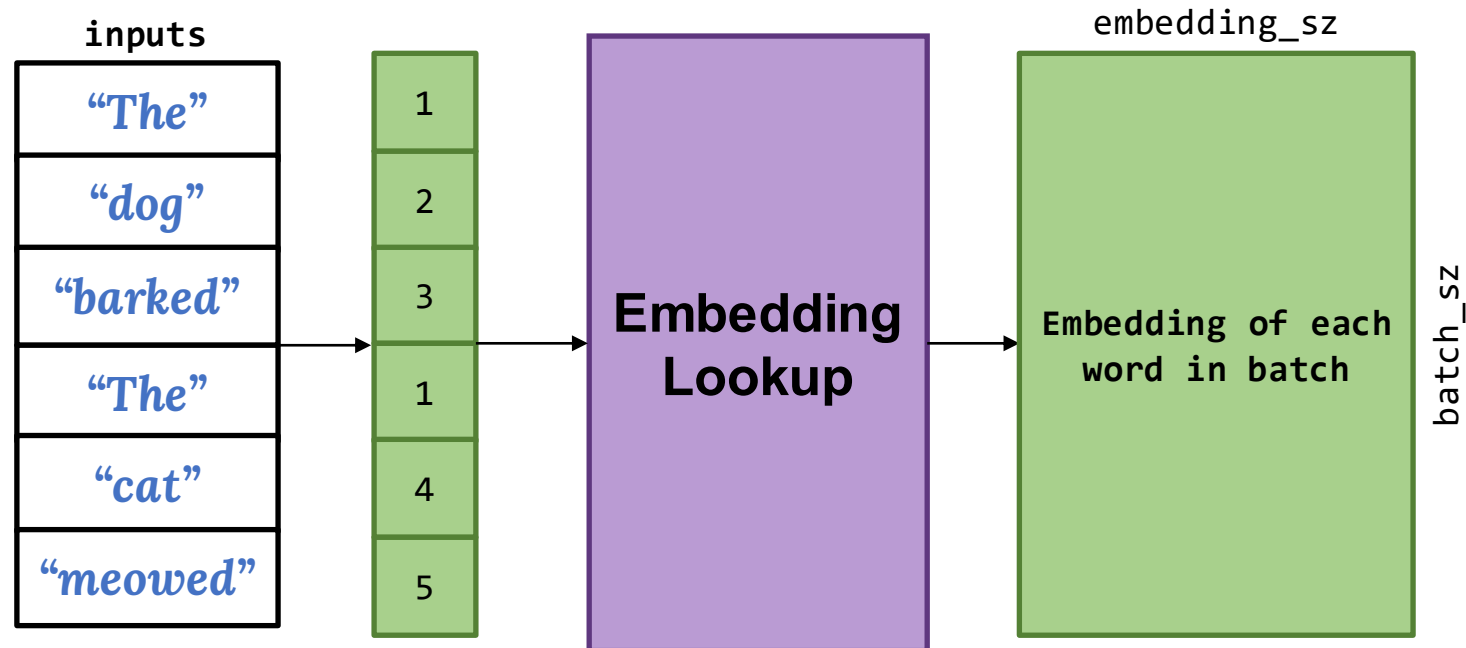
Size of Feed Forward bigram Model

Let's look at bigram model and count the number of weights.



Size of Feed Forward bigram Model

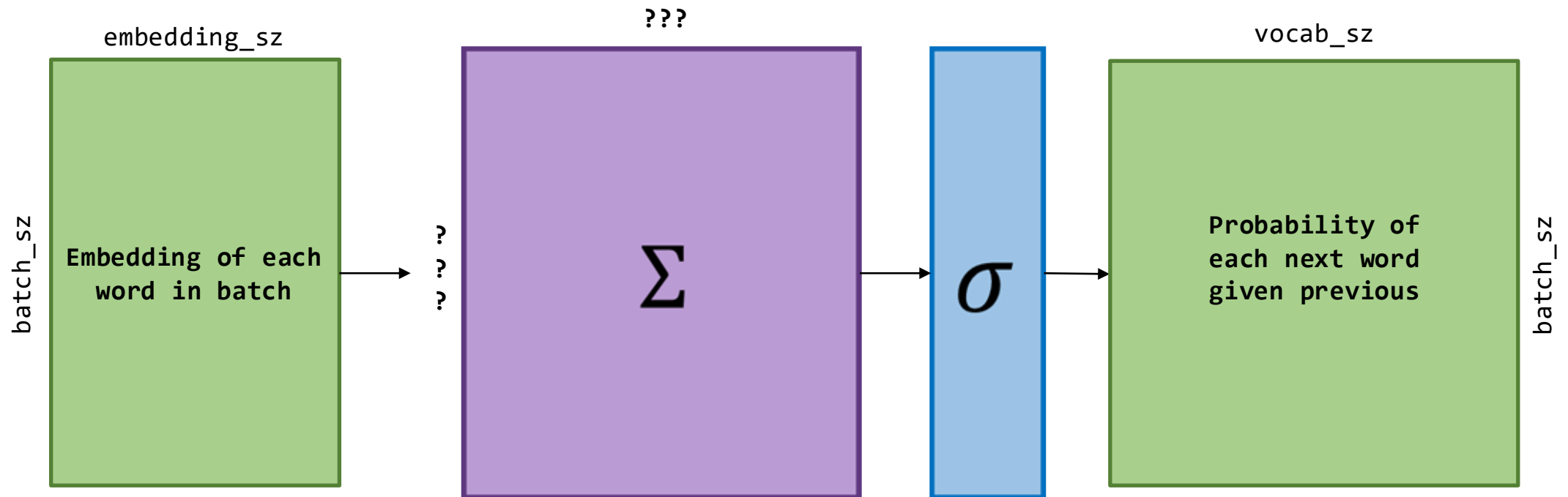
To preform embedding lookup on our entire batch, we just need one embedding matrix of size: (vocab_sz, embedding_sz)



Size of Feed Forward bigram Model

What size do we need the linear layer to be in order to map:

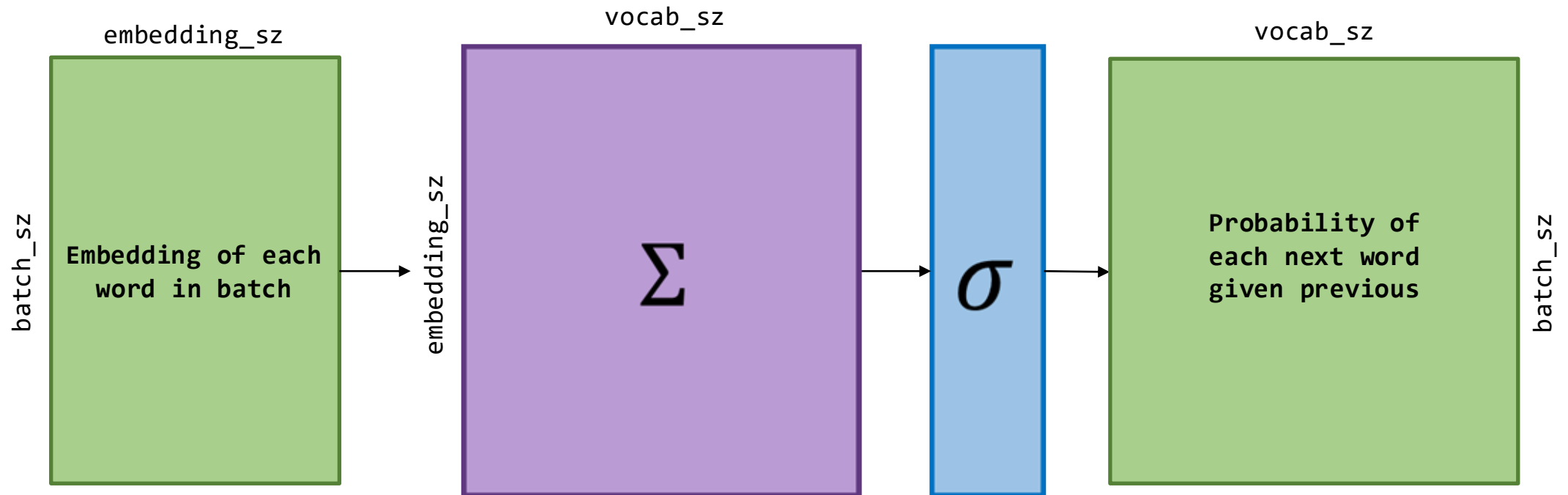
$$(\text{batch_sz}, \text{embedding_sz}) \times (???, ???) \rightarrow (\text{batch_sz}, \text{vocab_sz})$$



Size of Feed Forward bigram Model

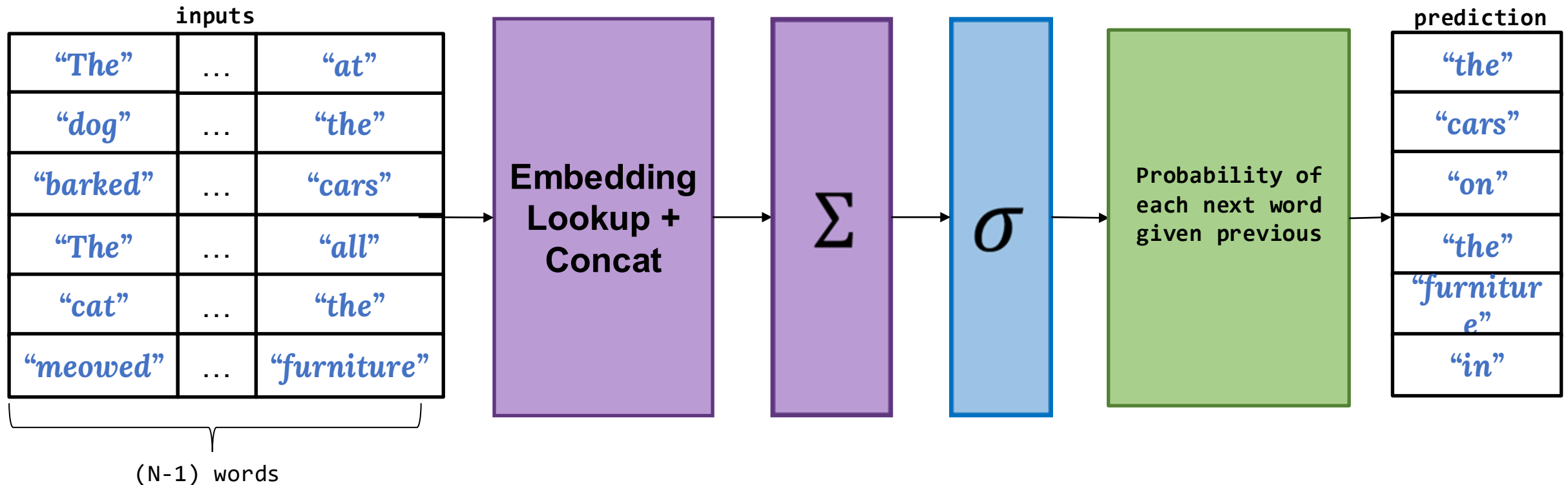
What size do we need the linear layer to be in order to map:

$$(\text{batch_sz}, \text{embedding_sz}) \times (???, ???) \rightarrow (\text{batch_sz}, \text{vocab_sz})$$



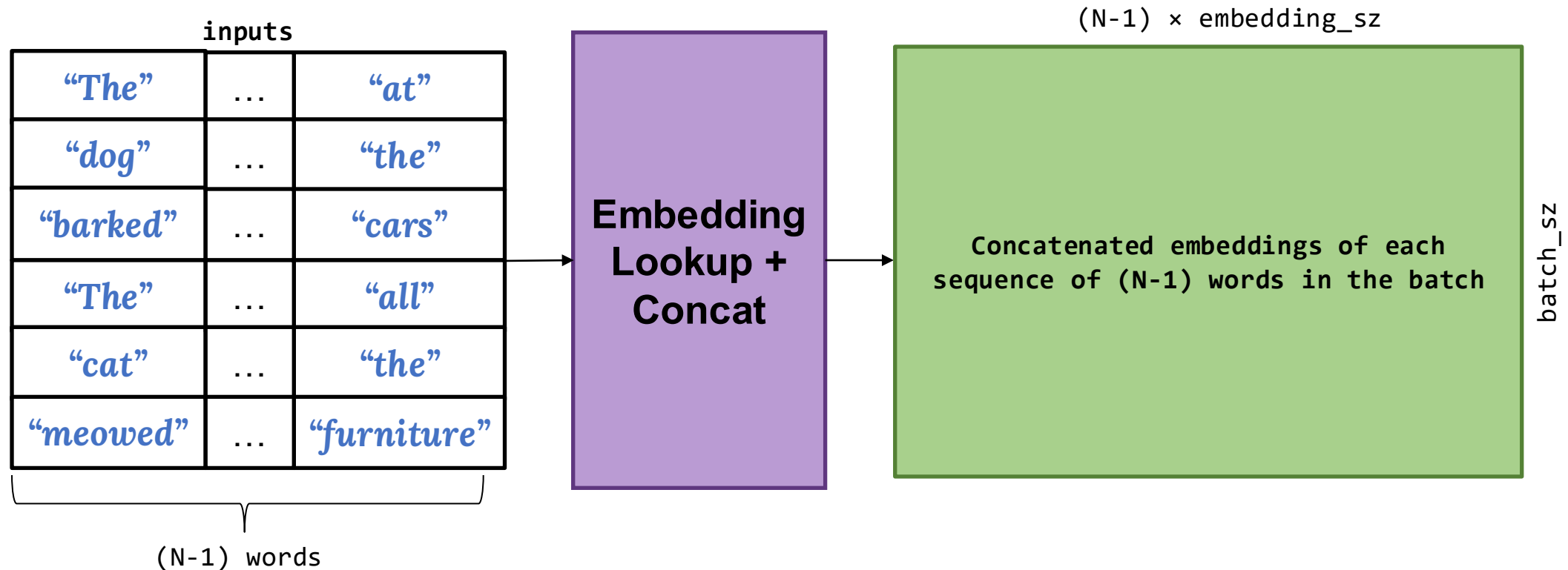
Size of Feed Forward N-gram Model

So what happens in the N-gram case?



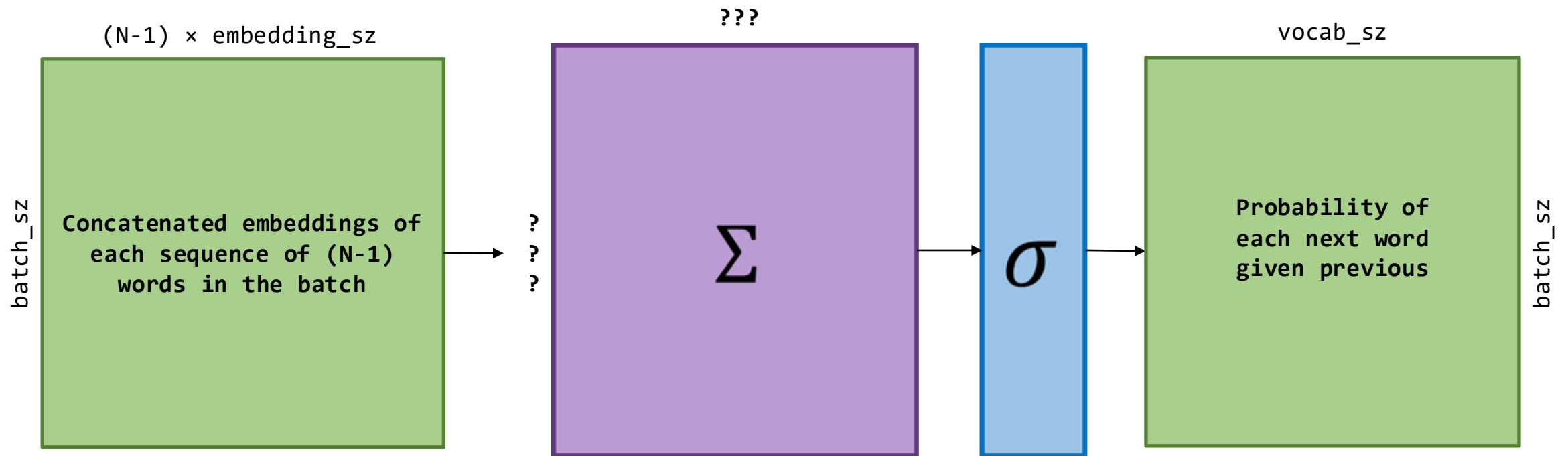
Size of Feed Forward N-gram Model

Embedding lookup + Concatenation still requires only one embedding matrix of size: (vocab_sz, embedding_sz)



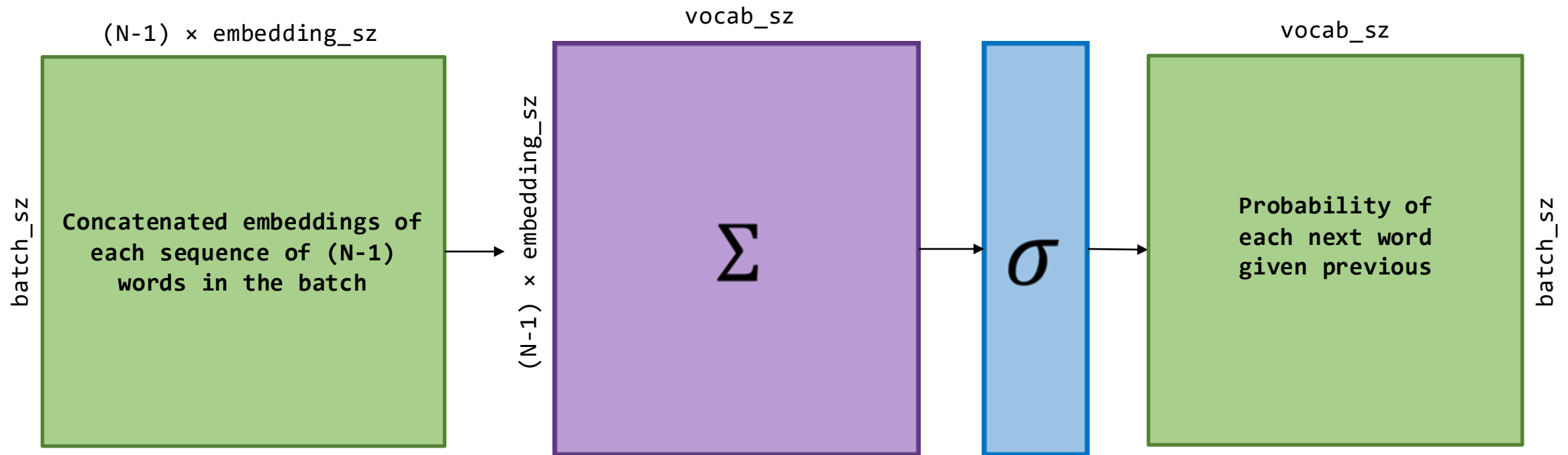
Size of Feed Forward N-gram Model

But what happens to our feed forward layer?



Size of Feed Forward N-gram Model

It needs to be size: $((N-1) \times \text{embedding_sz}, \text{vocab_sz})$
For every word, we add $(\text{embedding_sz} \times \text{vocab_sz})$ more weights!



Wrap up



What was the clearest point today?

What was the muddiest point today?

